Week 11 Notes

Longest Paths in a DAG: Scheduling, Transitive Closure, Matrices, Weighted Graphs, Shortest Paths and Spanning Trees

Course: Mathematics–1

Notes created by: Vishal (Instructor)

Lecture 1 — Longest Paths in a DAG

Doing tasks in parallel; the recurrence; computing longest paths alongside the topological sort; why cyclic graphs are hard

1 The second problem on a DAG

We have been looking at directed acyclic graphs as representations of sets of tasks with dependencies, and we said there are **two natural problems**:

- find a **feasible sequence** in which the tasks can be done — this was the **topological sort**;
- find out **how many steps** we need to perform the tasks in an **optimal** way.

A topological sort lists the vertices so that whenever there is an edge from i to j, i appears before j, giving a feasible schedule. Now we ask how *fast* we can do this, if tasks with no dependencies between them **can be done together**.

Sequence versus parallelism

In a topological sort we would have to put 0 before 1, or 1 before 0. But if there are no dependencies between them and it is possible to do them in parallel, then we could do 0 and 1 **at the same time** — provided there are resources for both.

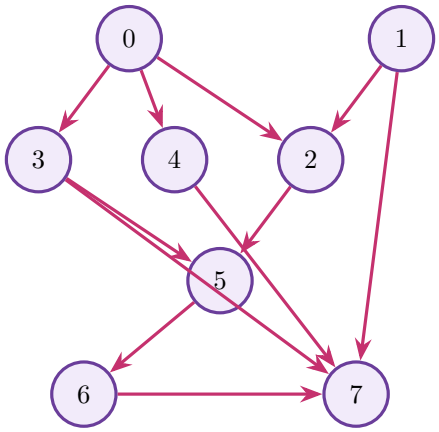
Example 1.1 — taking courses

When you take courses you do not do just one course in a semester; you can do several. As long as the number available to you is reasonable, you might do three, four or five in a semester. So if the DAG represents **prerequisites** between courses, and each course takes a semester — so you can only go on to the next course in the next semester — the natural question is:

how many semesters do I need to complete the remaining requirements?

2 The example graph

Here is the DAG we shall work with, on eight vertices 0 to 7:



Treating these as courses:

Scheduling the courses semester by semester

Semester	Courses	Why
1	0, 1	no prerequisites at all
2	2, 3, 4	everything they need is done
3	5	its prerequisites 2 and 3 are now done
4	6	cannot start until 5 is finished
5	7	cannot start until 6 is finished

We get stuck in the sense that we cannot do 7 until 6 is finished, and cannot do 6 until 5 is finished. So with this DAG as our prerequisite structure, we **cannot do better than five semesters**. This is the quantity we want to compute.

3 Reducing it to longest paths

Formally, this consists of finding the **longest path** in the DAG.

> The strategy is to compute the longest path **to every vertex**. Once we have those, the **maximum** among them is the longest path overall.
> Asking "when can I do course 6?" or "when can I do course 3?" for each vertex, and then taking the maximum, tells us the total: if some course requires the fifth semester, then overall we need five semesters.

We take the longest path to an **initial** vertex to be 0 — in the course example, how many semesters we have to wait before taking it. With 0 waiting we can do anything of **indegree** 0, so those are a good starting point. In our graph, vertices 0 and 1 are exactly those.

4 The recurrence

Now what happens for a vertex whose indegree is not 0? Take vertex 5: it can happen only after both 2 and 3 have happened. So if the longest path to 2 is k and the longest path to 3 is l, then we must wait for

$$\max(k, l),$$

because 2 and 3 cannot both be finished before then.

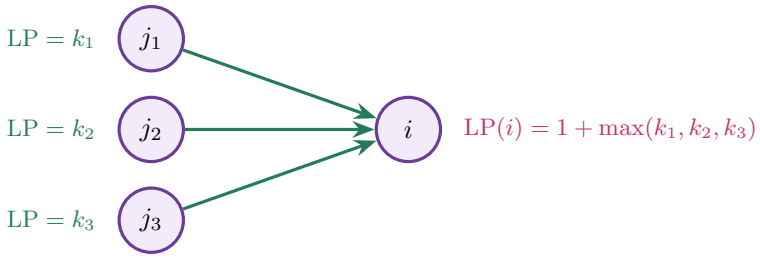
The illustration

Suppose one of them happens in the **fourth** semester and the other in the **fifth**. Then only in the **sixth** semester can we get to course 5 — that is, the maximum, plus one.

The longest-path recurrence

$$\boxed{\mathrm{LP}(i) \;=\; 1 \;+\; \max\Big\{\,\mathrm{LP}(j) \;:\; (j,i) \in E\,\Big\}}$$

That is: take the set of all longest-path values $\mathrm{LP}(j)$ over the **incoming** neighbours j of i, take the maximum of all of those, and **add** 1, because we now have to go to the next semester.



5 Why we compute it in topological order

To compute $\mathrm{LP}(i)$ we need to know the longest paths for **all** the incoming neighbours. How do we know those? We know them if we have **already calculated** them — and we will have calculated them if we go in **topological order**.

If we sort the vertices according to the dependencies, then by the time we come to i, all the incoming neighbours of i have **already been listed** before i. So if we compute the longest path as we list, that information is available exactly when we need it.

5.1 Stated formally

Suppose

$$i_0,\; i_1,\; \ldots,\; i_{n-1}$$

is a topological ordering of V — a rearrangement of the numbers 0 to $n-1$ into some sequence. If we look at a particular entry i_k, then **all its incoming neighbours in the graph must appear before it**. So if we compute **from left to right**, then for each i_k we can compute the longest path based on values already computed to the left.

Since we do this as we go along, we do **not** have to enumerate all the vertices first. By the time we reach a vertex we already have the information to compute its longest path — so we can **overlap** the topological sort and the longest-path computation and do them **at the same time**.

6 The algorithm

As before we compute the **indegree** of every vertex, and we also **initialise the longest path to** 0 for all vertices, because initially we know nothing.

Throughout the trace we keep two numbers per vertex:

- the **blue** number — our **current knowledge** about the longest path;
- the **red** number — the **indegree**, which we are using for the topological sort.

Why partial values are safe — the monotonic property

The maximum has a **monotonic** property: if we take the maximum of a set, it can only *increase* as we go along; it can never go back down. So once we have **frozen** the value for an incoming vertex — say vertex 1 with longest path 0 — we already know that the value for 2 must be at least $1 + 0$, and we can keep updating from there. In this way the maximum is computed **incrementally**.
(Here 0 is a particularly simple starting value, because a path cannot have a negative length, so the maximum can never fall below 0.)

7 The full trace

The initial state: indegrees from the columns, and all longest paths at 0.

Initial state

vertex	0	1	2	3	4	5	6	7
indegree	**0**	**0**	2	1	1	2	1	4
longest path	0	0	0	0	0	0	0	0

We follow the **same order as last time**, starting with vertex 1, which has indegree 0.

Topological sort and longest path, computed together

Step	Output	Indegrees reduced	Longest paths updated
1	1 (LP 0)	$2 \to 1$, $7 \to 3$	$2 : 0 \to 1$, $7 : 0 \to 1$
2	0 (LP 0)	$2 \to 0$, $3 \to 0$, $4 \to 0$	$3 : 0 \to 1$, $4 : 0 \to 1$; 2 stays 1 (since $1 + 0 = 1$)
3	3 (LP 1)	$5 \to 1$, $7 \to 2$	$5 : 0 \to 2$, $7 : 1 \to 2$
4	2 (LP 1)	$5 \to 0$	5 stays 2 (since $1 + 1 = 2$)
5	5 (LP 2)	$6 \to 0$	$6 : 0 \to 3$
6	6 (LP 3)	$7 \to 1$	$7 : 2 \to 4$
7	4 (LP 1)	$7 \to 0$	7 stays 4 — **no change**
8	7 (LP 4)	—	**done**

7.1 Reading the trace

- **Output 1.** Two things happen: we update the indegree of everything 1 points out to, *and* we now know definitive information about vertex 1. Its edges go to 2 and 7, so the indegree of 2 falls from 2 to 1 and that of 7 from 4 to 3. For both, the longest path goes from 0 to 1, because we know it is **at least** 1.
- **Output 0.** We update the indegrees of the three vertices it points to, and also their longest paths. Note that vertex 2 *already* believes its longest path is 1; taking the new information from

0 gives $1 + 0 = 1$, so **nothing changes**. But 3 and 4, which previously believed 0 without any justification, are now updated to 1.

- **Output 3.** This updates 5 and 7. Vertex 7 believed its longest path was 1, but now it must be at least $1 + 1 = 2$, so it changes. And 5 goes **directly from** 0 **to** 2 — notice this is not simply incrementing: it is computing $1 + \max$, and the maximum known incoming value is now 1.
- **Output 2.** Since $1 + 1 = 2$, nothing happens to the longest path of 5; only its indegree reduces.
- **Output 5.** This influences 6: the incoming edge to 6 has longest path 2 at the other end, so the path to 6 must be at least $2 + 1 = 3$. The indegree of 6 falls from 1 to 0, and its longest path goes from 0 to 3.
- **Output 6.** The indegree of 7 goes to 1, and 7 **jumps from** 2 **to** 4, because we have discovered this path of length 3 coming up to 6, one of its incoming neighbours. But we cannot enumerate 7 yet, since its indegree is not 0.
- **Output 4.** Now the indegree of 7 becomes 0, but there is **no change** to its longest path — the longest path does not come through 4; it already came through 6, and 7 has already discovered that. We simply could not *finalise* it until reaching the enumeration of 7.
- **Output 7.** And now we have the longest path for every vertex.

topological order	1	0	3	2	5	6	4	7
longest path	0	0	1	1	2	3	1	**4**

Indegree decrements versus maxima

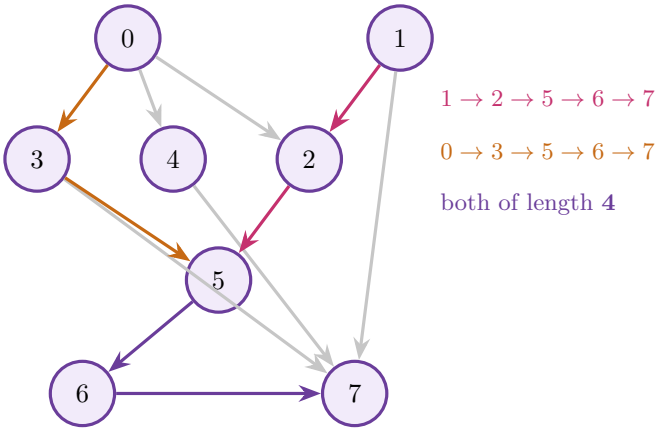
We are overlapping two computations. In the **topological sort** we **decrement** the indegree each time we remove an edge. In the **longest-path** computation we take a **maximum** — so sometimes the value changes and sometimes it does not, and when it does change it may change by **more than** 1. We must keep track of the largest incoming value seen so far and add 1 to it.

8 Checking the answer on the graph

Going back to the graph, we can verify that the longest path to 7 has length 4. One such path is

$$1 \to 2 \to 5 \to 6 \to 7,$$

which is four edges. The longest path is measured in the **number of edges** — how many things must be done before we get here, that is, how many semesters we must work before taking course 7.



In general there are **multiple longest paths**. Here we can also take the path $0 \to 3 \to 5 \to 6 \to 7$, which is also of length 4. So the fact that we have *a* longest path does not mean it is a **unique** one.

9 Summary of the two DAG problems

Directed acyclic graphs are a natural way to represent dependencies.

- The **topological sort** gives a feasible schedule — a sequence in which the tasks can be done such that all dependencies are satisfied before we reach a task.
- The **longest path** gives the **shortest duration** needed if we are allowed to do tasks in parallel.

The longest path can be computed **in an overlapping way** with the topological sort: once we have processed all the dependent vertices of a vertex, we have enough information to process that vertex itself. So in the same sequence in which we enumerate, we can associate with each vertex its longest path and build this up incrementally — **without any extra work**.

10 What about graphs with cycles?

One might ask whether the longest path makes sense in a graph **with** cycles — because with cycles we can go round and round: visit a vertex, come back to it, and go on again.

But recall the distinction between a **path** and a **walk**: a path cannot repeat a vertex, whereas a walk can. Going around a cycle and then proceeding is technically a **walk**, not a path.

So with the literal interpretation of *path*, longest paths in cyclic graphs **do** make sense: what is the longest sequence of edges we can travel before repeating a vertex? This can be at **most** $n-1$ — but **is** it $n-1$? That is the question.

10.1 The surprising difficulty

In a directed acyclic graph we came up with a **reasonably efficient** algorithm, which processes each vertex only once; then, depending on whether we represent it as an adjacency list or an adjacency matrix, we do some scanning to find the incoming and outgoing edges. Beyond that it is very reasonable — it takes time proportional to n^2, or to $n+m$, depending on the representation.

In the **general case** it turns out to be **surprisingly hard**. If the graph has cycles we can certainly define the longest path by considering only paths in which vertices do not repeat — so we know it is at most $n-1$ — but to know *whether* it is $n-1$ or not is surprisingly hard.
In fact there is **no known efficient way** to do this. Of course it can be done by **brute force**, by examining every possible path in the graph, counting its length, and taking the maximum — and what we do not know at present is whether there is any better way.

So there is a **huge gap** between the longest-path problem on DAGs and the longest-path problem on directed graphs which are *not* DAGs. On non-DAGs it is very difficult computationally, compared with the relatively simple problem we have on DAGs.

Lecture 2 — Transitive Closure

Relations as graphs; the ancestor relation; paths of length ℓ as matrix powers; computing A^+

1 Back to relations

One of our original motivations for looking at graphs was to **visualise relations**. So let us go back to relations.

Relation

A relation R on a set S is a **subset of the Cartesian product** $S \times S$. That is, $S \times S$ is all pairs (s_1, s_2) taken from S, and some subset of these forms the relation.

Example 2.1 — the parent relation

Suppose we have a set of people — perhaps a family — and we want to describe the family tree. We can represent as a relation the fact that two people are related as **parent and child**:

$$(p, q) \in R \qquad \Longleftrightarrow \qquad p \text{ is a parent of } q.$$

2 The ancestor relation

Given the parent relation, a very natural question is: what is the **grandparent** relation, the **great-grandparent** relation, and so on — in general, the **ancestor** relation. Is p an ancestor of q? In the family tree, is q a descendant of p?

To find out, we have to **trace a sequence** of relationships: find a child of p, then a child of that child, and so on.

An ancestor sequence

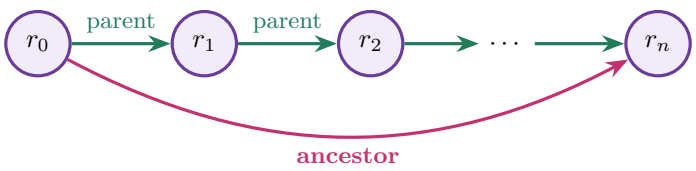
We look for a sequence of people

$$r_0,\ r_1,\ r_2,\ \ldots,\ r_n$$

such that

$$r_0 \text{ is a parent of } r_1, \quad r_1 \text{ is a parent of } r_2, \quad \ldots, \quad r_{n-1} \text{ is a parent of } r_n.$$

Then r_0 is an **ancestor** of everybody to the right in the sequence. In particular, if $r_0 = p$ and $r_n = q$, we have established that p is an ancestor of q.



This is a **new relation**: it relates pairs of people connected by a *sequence* of parent relations. And it has a name.

3 Transitive closure

Transitivity

A relation is **transitive** if

$$A \text{ related to } B \quad \text{and} \quad B \text{ related to } C \qquad \Longrightarrow \qquad A \text{ related to } C.$$

The parent relation is **not** transitive. But what happens if we *force* transitivity onto it — if we insist that whenever somebody is related to somebody and that somebody is related to a third person, the first is related to the third? That gives exactly the **ancestor** relation.

The transitive closure

When we **close** the parent relation under transitivity, we compute what is called the **transitive closure**, denoted

$$R^+.$$

Here R^+ means that we have to apply R **one or more times** to go from p to q.

Our question, of course, is: **how do we calculate this transitive closure systematically?**

4 From a relation to a graph

Recall that we can represent any such relation as a **directed graph**. The vertices represent the set S, so $V = S$, and each edge represents a pair in the relation:

$$(u, v) \in R \qquad \Longrightarrow \qquad \text{put an edge from } u \text{ to } v,$$

and this is the **only** case in which we put an edge from u to v. So the edges are exactly the pairs related by R.

What we defined as R^+ can be calculated in the graph as being a **path**.

So, for instance, in our earlier airline network we can say that v_9 is related by R^+ to v_0, because there is a **path** from v_9 to v_0.

4.1 The first strategy: repeated BFS or DFS

How do we find these paths? We want *all* such pairs, and rather than focusing on a single path we can compute **reachability** — what is reachable from every vertex — using breadth-first or depth-first search.

For R^+ we are interested in reachability **for every** i **and every** j: we want to know, for each pair, whether it falls into R^+. So we must compute reachability from *every* starting point:

Start at vertex 0 and perform BFS — now we know everything of the form $(0, j)$. Then start at 1 and perform BFS again, and so on.

Note that this is a **directed** graph, so we must do it from every vertex: if 0 can reach j, it is neither obvious nor required that j can reach 0.

5 The second strategy: the adjacency matrix

Here is another strategy. In the adjacency matrix we put a 1 if there is an edge from i to j. But another way of thinking of an edge is that

a single edge is a **path of length** 1 from i to j.

Since R^+ is also a relation, it corresponds to some new graph G^+ — the same set of vertices, but now with an edge whenever there is an R^+ relationship between i and j. And G^+ corresponds to an adjacency matrix A^+.

Our goal

$$A^+[i][j] = 1 \quad \Longleftrightarrow \quad \text{there is a path of length } \geq 1 \text{ from } i \text{ to } j.$$

A parent is an ancestor (a path of length 1); a parent's parent is an ancestor (length 2); a parent's parent's parent is an ancestor (length 3); and so on. **We want to compute A^+ from A.**

We know one way already — do BFS from every vertex and fill in A^+ from the results. But the question is: **can we do it directly, using just A?**

6 Paths of length two

The matrix A records paths of length 1; we may think of it as A^1. As a first step, let us compute paths of length **two**. A path of length 2 goes

$$i \longrightarrow k \longrightarrow j.$$

A path of length 2 can return to its start

A path of length 1 cannot go from i back to i — but a path of length 2 **can**.

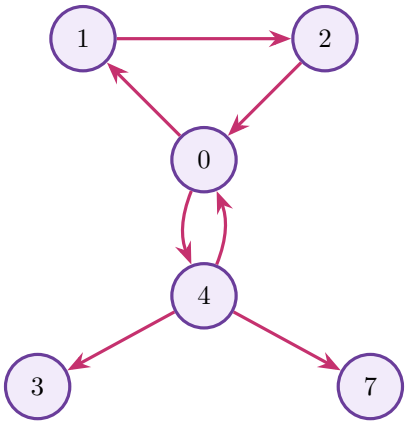
The rule for A^2

$$A^2[i][j] = 1 \quad \text{if there is some } k \text{ with } A[i][k] = 1 \text{ and } A[k][j] = 1.$$

That is: a path of length 1 from i to k, and a path of length 1 from k to j.

6.1 The worked example

Consider the following portion of the network, on vertices $0, 1, 2, 3, 4, 7$:



A — paths of length 1

	0	1	2	3	4	7
0	0	**1**	0	0	**1**	0
1	0	0	**1**	0	0	0
2	**1**	0	0	0	0	0
3	0	0	0	0	0	0
4	**1**	0	0	**1**	0	**1**
7	0	0	0	0	0	0

Filling in row 0 of A^2

Why is there an A^2 entry from 0 to 0? Because there is a $(0, 4)$ entry — a path of length 1 from 0 to 4 — and then a $(4, 0)$ entry. So choosing $k = 4$ with $i = 0$ and $j = 0$ gives $A^2[0][0] = 1$.

Doing it systematically for row 0:

Via k	Then to	Entry obtained
$0 \to 1$	$1 \to 2$	$A^2[0][2] = 1$; 1 has only that one outgoing edge, so nothing further through 1
$0 \to 4$	$4 \to 0$	$A^2[0][0] = 1$
	$4 \to 3$	$A^2[0][3] = 1$
	$4 \to 7$	$A^2[0][7] = 1$

The remaining rows

- **Row 1.** Vertex 1 has only one outgoing edge, to 2; and 2 has only one outgoing edge, to 0. So the only new thing discovered is $1 \to 2 \to 0$, giving $A^2[1][0] = 1$.
- **Row 2.** Vertex 2 has only one outgoing edge, to 0 — but 0 has outgoing edges to 1 and to 4. So we get $2 \to 0 \to 1$ and $2 \to 0 \to 4$, giving $A^2[2][1] = 1$ and $A^2[2][4] = 1$.

In this way, for every row we take the outgoing edges $i \to k$ and the outgoing edges $k \to j$, and add an entry from i to j. By scanning these two rows of the matrix we compute the whole of A^2.

A^2 — paths of length exactly 2

	0	1	2	3	4	7
0	**1**	0	**1**	**1**	0	**1**
1	**1**	0	0	0	0	0
2	0	**1**	0	0	**1**	0
3	0	0	0	0	0	0
4	0	**1**	0	0	**1**	0
7	0	0	0	0	0	0

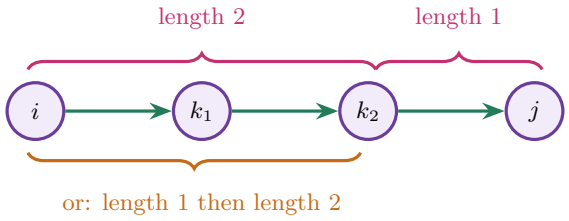
Paths of length 2 do not subsume paths of length 1. For instance, there *is* a path of length 1 from 0 to 1, but there is **no** path of length 2 from 0 to 1. These are paths of length *strictly* 2 — not of length 1. So A records the edges, and A^2 records paths of length 2: we can go from 0 to 0 in length 2, but the fact that we can go from 0 to 1 in length 1 does not mean we can do it in length 2.

7 Longer paths

How do we go to paths of length 3? A path of length 3 must be of the form

$$i \longrightarrow k_1 \longrightarrow k_2 \longrightarrow j,$$

so there must be two intermediate vertices. We can **split it** whichever way we want: either as a path of length 2 followed by a path of length 1, or as a path of length 1 followed by a path of length 2.



We already have explicit matrices for lengths 1 and 2, so:

The rule for A^3

$$A^3[i][j] = 1 \quad \text{if there is some } k \text{ with } A^2[i][k] = 1 \text{ and } A[k][j] = 1.$$

Earlier we looked inside A for two entries; now we look at an entry in A^2 and an entry in A, and try to **match them up** at a common k.

So we started with A, did one pass over A to get A^2; and now, using A^2 and A, we get A^3.

A^3 — paths of length exactly 3

	0	1	2	3	4	7
0	**1**	**1**	0	0	**1**	0
1	0	**1**	0	0	**1**	0
2	**1**	0	**1**	**1**	0	**1**
3	0	0	0	0	0	0
4	**1**	0	**1**	**1**	0	**1**
7	0	0	0	0	0	0

7.1 The general step

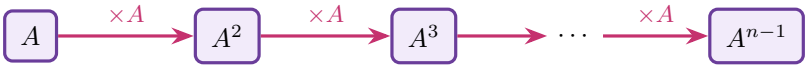
For a path of length 4 we decompose it as a path of length 3 followed by one edge, taking the entries in A^3 and combining them with entries in A. (We could equally do $1+3$ or $2+2$, but let us follow the general rule of breaking it into **one less, plus one**.)

The general recurrence

$$\boxed{A^{\ell+1}[i][j] = 1 \quad \text{if there is some } k \text{ with } A^{\ell}[i][k] = 1 \text{ and } A[k][j] = 1.}$$

We go from i to k in ℓ steps, and then from k to j in one step — so we go from i to j in $\ell + 1$ steps.

So we do the same thing again and again: first A combined with A, then A^2 combined with A, and in general A^ℓ combined with A — building a matrix that captures longer and longer paths.



8 Where do we stop?

Stop at A^{n-1}

If there is a path at all, it cannot have more than $n-1$ edges. Once we have traversed $n-1$ edges we have seen $n-1$ different vertices besides the starting point, so anything beyond that must **repeat** a vertex — and then there must have been a **shorter** path.

So once we have computed A^{n-1} we have everything of interest: all paths of length $1, 2, 3, 4, \ldots, n-1$. Any path longer than $n-1$ cannot contribute new information about whether i and j are connected by the relation.

9 Combining everything into A^+

Recall why we are doing all this: (i, j) is in the transitive closure R^+ if there is a path from i to j in the corresponding graph, and we have observed many times that the length of such a path is at most $n-1$.

We have the original A (which is A^1, paths of length 1), then A^2, then A^3, and so on up to A^{n-1} — that is $n-1$ matrices, giving all information about paths of lengths 1 through $n-1$. We want an edge from i to j in R^+ if there is an edge **somewhere in one of these**.

The formula

$$\boxed{A^+[i][j] \;=\; \max_{1 \le \ell \le n-1} A^{\ell}[i][j]}$$

9.1 Why the maximum does the job

Every entry of these matrices is 0 or 1 — either there is a path of that length or there is not.

- If **all** of them are 0 — no path of length 1, none of length 2, none of length 3, and so on — then the maximum is also 0, so $A^+[i][j] = 0$: there is no path.
- If there is a 1 **anywhere**, in any one of those ℓ positions from 1 to $n-1$, then the maximum becomes 1.
- If there are **many** paths — say one of length 3 and one of length 7 — it is still 1, because $\max(1, 1) = 1$.

So by taking the maximum we are simply recording: **is there at least one 1 in that sequence?** That is, across all these matrices, is there at least one with value 1 in position (i, j)?

$A^+[i][j]$ = 1 **if and only if** there is a path from i to j
— and such a path is always bounded in length by $n - 1$.

For our example graph, combining the matrices gives:

A^+ — the transitive closure

	0	1	2	3	4	7
0	1	1	1	1	1	1
1	1	1	1	1	1	1
2	1	1	1	1	1	1
3	0	0	0	0	0	0
4	1	1	1	1	1	1
7	0	0	0	0	0	0

The rows for 3 and 7 are entirely 0, because those vertices have **no outgoing edges** at all in this portion of the network, so nothing is reachable from them.

10 This is really matrix multiplication

What we have done can be reformulated in a way that is called **matrix multiplication**, which we will not do right now. But it is important to know that this seemingly tedious calculation — chasing rows and columns — is actually a very **standard operation on matrices**.

In this form we can write it as a sort of multiplication of matrices. It is not exactly the ordinary product as written here, but for our purposes this is fine. The reason we are doing this with matrices is precisely that the operation is a standard mathematical operation on matrices — so although we have done the row and column chasing explicitly, looking for a k here and a k there, it can be done directly as a matrix operation. It is therefore nothing new.

11 Summary

The **transitive closure** tells us whether there is a **sequence of intermediate elements** connecting two elements: we start at p and go through multiple R edges to reach q. Our example was the **ancestor** relation — a sequence of parent edges generates an ancestor edge.

Since we visualise relations as graphs, this corresponds to a **path**. In general these are **directed** edges, because such relations are not assumed to be symmetric: the parent relation certainly is not — if A is a parent of B, then B is clearly not a parent of A. So we follow a path in a directed sense, and this is just a **reachability** question in graphs.

Method	How it works
Repeated BFS / DFS	run the search from *every* starting vertex and record what is reachable
Matrix powers	take the adjacency matrix and do a form of matrix multiplication to go from A to A^2 to A^3 and so on, stopping at A^{n-1}; then look for a 1 in any one of those $n-1$ matrices, and if so declare $A^+[i][j] = 1$

We stop at A^{n-1} because it records paths of length $n-1$, which is the maximum path length useful to us in deciding whether two nodes are connected.

Lecture 3 — Matrix Operations

Matrices and their notation; addition; matrix multiplication; compatible shapes; Boolean matrices and transitive closure

1 Why matrices again

We have used matrices to represent graphs — the **adjacency matrix**. When we looked at the transitive closure problem, we suggested that it might be useful to describe the computation using some operations directly on matrices. So let us look at matrix operations, leading up to the notion of **matrix multiplication**.

What a matrix is

A matrix is a **two-dimensional table of values**, usually numbers. Being two-dimensional, it has **rows** and **columns**: say r rows and c columns, where r and c could be different numbers. We write this as $r \times c$, and call it an $r \times c$ matrix.
As usual we number the rows and columns **starting from** 0:

$$\text{rows } 0, 1, 2, \ldots, r-1, \qquad \text{columns } 0, 1, 2, \ldots, c-1.$$

In the adjacency matrix we have been looking at, $r = c = n$, the number of vertices — one row and one column for every vertex — so we number both from 0 to $n-1$. There the entries are 0 and 1, but in general they could be any numbers.

2 A different example: freight traffic

For a completely different example, suppose we want to record information about **freight traffic by rail** between cities. Take the six biggest cities in India — Bangalore, Chennai, Delhi, Hyderabad, Kolkata and Mumbai — and suppose we have the volume of traffic in, say, millions of tonnes.

Every city corresponds to a row and a column, so this is a **square** matrix with 6 rows and 6 columns. Numbering in alphabetical order — it does not really matter in what order:

0	1	2	3	4	5
Bangalore	Chennai	Delhi	Hyderabad	Kolkata	Mumbai

Two features of this matrix

- It is **not symmetric**. There is a certain amount of traffic from Bangalore to Chennai, but a different amount from Chennai to Bangalore — there is no reason why freight traffic should be the same in both directions.
- Along the **diagonal** there is no freight recorded, since traffic from a city to itself does not make sense for us.

3 Matrix addition

In India the financial year starts in April. So suppose that for the **first half** of the financial year, from 1 April to 30 September, we have one set of volumes, and for the **second half** we have another. The numbers are different.

A natural thing is to compute the freight volume for the **full year**. Clearly, for any pair of cities it is the **sum** of the two.

Example 3.1 — Delhi to Kolkata

Delhi is city 2 and Kolkata is city 4. Looking up the entry for the first half of the year gives 595; the second half gives 326. Put together,

$$595 + 326 = 921.$$

First half year A and Second half year B

A	0	1	2	3	4	5
0	—	412	268	355	247	503
1	388	—	301	274	190	462
2	254	317	—	286	**595**	640
3	361	259	273	—	208	431
4	233	176	549	221	—	394
5	517	448	662	407	372	—

B	0	1	2	3	4	5
0	—	375	294	318	399	466
1	402	—	288	315	224	419
2	279	342	—	264	**326**	587
3	330	296	251	—	236	408
4	262	205	512	249	—	377
5	484	421	619	388	345	—

Notation

Rows are numbered 0 to $r-1$ and columns 0 to $c-1$. The entry where the i-th row meets the j-th column is written

$$M[i][j] \qquad \text{or} \qquad M_{ij}.$$

We want to compute the aggregate volume for the whole year into a new matrix C:

$$C_{ij} = A_{ij} + B_{ij}, \qquad \text{written concisely as} \qquad \boxed{C = A + B}$$

The condition for adding

When we have matrices A and B of the **same size** — the same number of rows and the same number of columns — then for every position in A there is a corresponding position in B. So we can add those entries and put the result into a third matrix of the same size. We are doing this in the same way for **every** element of the matrix.

4 A first guess at multiplication

We were not so interested in adding matrices but in **multiplying** them. Can we multiply them the same way? The obvious rule would be: instead of adding A_{ij} to B_{ij}, **multiply** A_{ij} by B_{ij}.

Element-wise product — the obvious guess

$$A = \begin{pmatrix} 2 & 7 & 1 \\ \mathbf{4} & 3 & 9 \\ 6 & 2 & 8 \\ 1 & 5 & \mathbf{5} \end{pmatrix} \qquad B = \begin{pmatrix} 3 & 1 & 6 \\ \mathbf{5} & 2 & 4 \\ 7 & 9 & 2 \\ 8 & 3 & \mathbf{4} \end{pmatrix} \qquad C = \begin{pmatrix} 6 & 7 & 6 \\ \mathbf{20} & 6 & \mathbf{36} \\ 42 & 18 & 16 \\ 8 & 15 & \mathbf{20} \end{pmatrix}$$

For instance $C_{10} = A_{10} \times B_{10} = 4 \times 5 = 20$; similarly $A_{32} = 5$ and $B_{32} = 4$, so again 20; and $9 \times 4 = 36$. We have just multiplied entries one at a time.

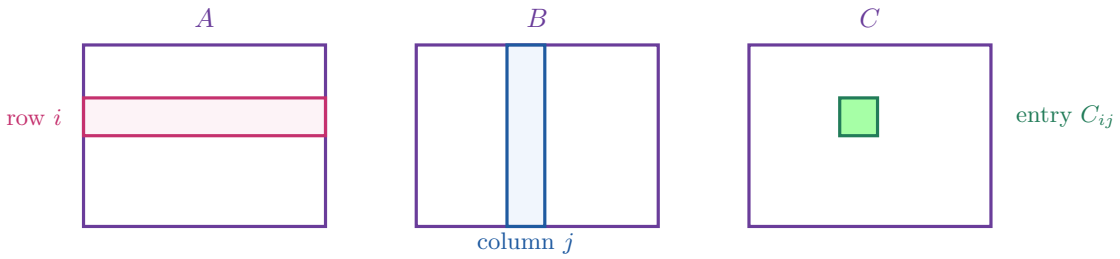
There is **nothing wrong** with this way of manipulating matrices — it just does not turn out to be very useful in practice. Addition made sense for us; multiplication like this usually does not give us any useful number. So there is a **different** operation, given the name **matrix multiplication**, which is a little more complicated.

5 Matrix multiplication

We want to compute the product $A \times B$, so we need a rule for the (i, j)-th entry of the product.

The rule

To compute C_{ij}: look at **row i of A** and **column j of B**.
Take the first entry of that row, A_{i0}, and the first entry of that column, B_{0j}, and **multiply**. Move right to the next entry of the row, A_{i1}, and down to the next entry of the column, B_{1j}, and multiply. Keep walking along the row of A while walking down the column of B, multiplying the pair at every step — then **add all these products up**.



It seems really complicated, but this is how matrix multiplication works.

5.1 A worked example

$A \times B = C$

$$A = \begin{pmatrix} 2 & 1 & 3 & 0 \\ 4 & 0 & 9 & 7 \\ 6 & 3 & 0 & 1 \\ 1 & 5 & 2 & 4 \end{pmatrix} \qquad B = \begin{pmatrix} 1 & 1 & 2 & 8 \\ 0 & 0 & 4 & 2 \\ 3 & 0 & 1 & 1 \\ 2 & 6 & 0 & 0 \end{pmatrix} \qquad C = \begin{pmatrix} 11 & 2 & 11 & 21 \\ 45 & \mathbf{46} & 17 & \mathbf{41} \\ 8 & 12 & 24 & \mathbf{54} \\ 15 & 25 & 24 & 20 \end{pmatrix}$$

Computing C_{13}. Look at row 1 of A and column 3 of B:

$$C_{13} = \underbrace{4 \times 8}_{32} + \underbrace{0 \times 2}_{0} + \underbrace{9 \times 1}_{9} + \underbrace{7 \times 0}_{0} = 32 + 9 = \boxed{41}$$

The zeros are nice — they cost us nothing.

Computing C_{11}. Row 1 of A with column 1 of B:

$$C_{11} = 4\times 1 \;+\; 0\times 0 \;+\; 9\times 0 \;+\; 7\times 6 \;=\; 4+42 \;=\; \boxed{46}$$

Computing C_{23}. Row 2 of A with column 3 of B:

$$C_{23} = 6\times 8 \;+\; 3\times 2 \;+\; 0\times 1 \;+\; 1\times 0 \;=\; 48+6 \;=\; \boxed{54}$$

How much work is this?

That is just **one** entry, and we have to do it for every position. For each position in the output we scan a row of the first matrix and a column of the second, multiply each individual pair, and add them all up. So there is a lot of work to compute one entry.

5.2 The formula

For a square matrix, with $r = c = n$:

$$C_{ij} = A_{i0}B_{0j} + A_{i1}B_{1j} + \cdots + A_{i,n-1}B_{n-1,j},$$

using the last column of A and the last row of the column j of B. There is a simple way to write this: replace the whole summation by the summation sign, letting the second position range from 0 to $n-1$:

$$\boxed{C_{ij} \;=\; \sum_{k=0}^{n-1} A_{ik}\,B_{kj}}$$

This is just a shortcut for writing that long sum without specifying every term and writing dots.

6 When can we multiply? Compatible shapes

In general we do **not** need the matrices to be square. What we need is for the summation to be **well defined**: while walking along row i of A, we must be able to take the *same number of steps* down column j of B. So for every element we process in the row there must be a matching element to multiply it with.

- The **length of a row** in A is the **number of columns**, c_1 — it tells us how far we can go along a row.
- The **height of a column** in B is the **number of rows**, r_2 — going from the top row to the bottom row.

So what we need is

$$\boxed{c_1 = r_2.}$$

That is: if A is $m \times n$, then n must be the number of **rows** in B, and B can have any number of columns. We can take two matrices of different shapes, provided the number of columns in A equals the number of rows in B — only then can we do the summation correctly.

Example 3.2 — a 3×4 times a 4×3

$$P = \begin{pmatrix} 6 & 6 & 3 & 0 \\ 2 & 1 & 4 & 5 \\ 7 & 0 & 2 & 3 \end{pmatrix}, \qquad Q = \begin{pmatrix} 5 & 2 & 1 \\ 8 & 0 & 4 \\ 2 & 7 & 0 \\ 0 & 3 & 6 \end{pmatrix}$$

Here P has 3 rows and 4 columns, and Q has 4 rows and 3 columns. The first entry is

$$6 \times 5 + 6 \times 8 + 3 \times 2 + 0 \times 0 \;=\; 30 + 48 + 6 \;=\; \boxed{84}$$

and the full product is

$$PQ = \begin{pmatrix} \mathbf{84} & 33 & 30 \\ 26 & 47 & 36 \\ 39 & 37 & 25 \end{pmatrix}.$$

We end up with a matrix having as **many rows as** A and as **many columns as** B:

$$(m \times n) \;\times\; (n \times p) \;=\; (m \times p).$$

In our own context this does not matter, because with adjacency matrices we are always dealing with **square** matrices — but matrix multiplication in general does not require squareness, only this correspondence between the columns of A and the rows of B.

7 Back to transitive closure

Recall the problem that motivated all this. We started with a relation and wanted to know whether two objects are related by a **sequence** of pairs in the relation. We modelled it as a graph, with every pair in the relation becoming an edge. So finding a sequence of relational pairs is the same as finding a **sequence of edges** — the **reachability** problem.

The adjacency matrix captures paths of length 1: the direct relationships, which are given to us. And A^2 captures pairs connected by a path of length 2 — there must be an intermediate vertex k with $i \to k$ and $k \to j$.

So how do we put this in the framework of matrix multiplication?

8 Boolean algebra

We are not doing multiplication and addition over ordinary numbers. We are doing them over the values **true** and **false** — what is called **Boolean algebra**.

The entries indicate whether there is an edge:

$$1 = \textbf{true} \quad \text{(there is an edge)}, \qquad 0 = \textbf{false} \quad \text{(there is no edge)}.$$

So if we ask "is there an edge from 1 to 5?" and the entry is 0, the answer is *no* — *false*. If we ask "is there an edge from 3 to 4?" and $A_{34} = 1$, the answer is *true*. We work with these two values only.

The operations we are interested in over Boolean values are **AND** and **OR**, and we now replace them by arithmetic operations.

8.1 OR becomes +

The OR of two Boolean values is true provided **at least one** of them is true; only if both are false is the answer false.

OR, written as +

$$\begin{aligned} 0+0&=0 \quad \text{(false or false is false)}\\ 0+1&=1 \quad \text{(false or true is true)}\\ 1+0&=1 \quad \text{(true or false is true)}\\ \mathbf{1+1}&\mathbf{=1} \quad \textbf{(true or true is true)} \end{aligned}$$

The surprising line

Most of these follow from ordinary integer arithmetic: $1+0=1$ and $0+1=1$. What is perhaps surprising from a numeric point of view is that $\mathbf{1+1=1}$. Thinking of these as integers, $1+1$ would be 2 — but 2 is not a value we are dealing with; we have only true and false. So the line is saying **true or true is true**.
This is a strange kind of arithmetic, but it is justified because the underlying interpretation is in terms of **logical values and logical operations**.

8.2 AND becomes ×

AND is symmetric to OR in the sense that here we need **both** to be true, so the only interesting case is 1×1.

AND, written as ×

$$\begin{aligned} 1\times 1&=1 \quad \text{(true and true is true)}\\ 1\times 0&=0\\ 0\times 1&=0\\ 0\times 0&=0 \quad \text{(false and anything is false)} \end{aligned}$$

So if either of them is 0, the answer is false — whether both are false or only one is.

9 A^2 is a matrix product

Now look at what our rule says: $A^2_{ij}=1$ if for **some** k we have $A_{ik}=1$ and $A_{kj}=1$. What are the possible values of k? It must be one of the vertices, so k could be 0, or 1, or 2, up to $n-1$. Written out, that sentence becomes

$$(A_{i0} \text{ AND } A_{0j}) \text{ OR } (A_{i1} \text{ AND } A_{1j}) \text{ OR } \cdots \text{ OR } (A_{i,n-1} \text{ AND } A_{n-1,j})$$

If any of these pairs is simultaneously true, then there is an edge from i to j in A^2. If more than one is true it is still true — if there are multiple ways of going from i to j in two steps, that is fine; we need at least one.

Now replace every AND by × and every OR by +:

$$A^2_{ij} \;=\; \sum_{k=0}^{n-1} A_{ik}\,A_{kj}.$$

This is **exactly** the equation we wrote for an arbitrary matrix product. So

$$\boxed{A^2 = A \times A}$$

provided we think of A as having Boolean values 0 and 1, with OR as $+$ and AND as $\times$.

10 Higher powers

For paths of length 3: a path of length 3 breaks up as a path of length 2 followed by a path of length 1 — go from i to k in two steps, then from k to j in one. By the same logic,

$$A^3_{ij} = \sum_{k=0}^{n-1} A^2_{ik} A_{kj},$$

because A^2 captures the length-2 paths and A the length-1 paths.

Either decomposition works

We could equally decompose it as a length-1 path from i to k and a length-2 path from k to j. So A^3 is either $A^2 \times A$ or $A \times A^2$ — it does not matter.
Notice that for A^2 we had **no option**, because both factors are A: there is only one way to decompose a length-2 path, as $1+1$, since that is the only information we had. But for a length-3 path we can do $1+2$ or $2+1$.

In general, if we already know paths of length ℓ, we extend to $\ell+1$ by multiplying once more by A:

$$\boxed{A^{\ell+1} = A^\ell \times A}$$

We could also invert it — take one step first and then ℓ steps — and indeed if ℓ were 7 we could take 3 steps and then 4. But it is convenient to write it in a **uniform way**, so that we know how to proceed systematically from $1, 2, 3$ onwards. Each time we take the matrix computed at the previous step and multiply once more by A, so implicitly this is $A \times A \times A \times \cdots$ repeated ℓ times, and then one more.

11 Summing up to get the transitive closure

This gives us paths of length exactly 1, 2, 3 and so on — but what we wanted was whether there is a path of **any** length. And because it is a graph, if there is a path at all there must be one of length at most $n-1$: if it were longer there would be a loop we could remove, since there are only n vertices.

Why $n-1$

Starting from a vertex and taking $n-1$ steps, each time to a new vertex, there are only $n-1$ new vertices to go to. If we take one more step, the next vertex must be one of those already seen — and that would be useless.

So it suffices to look at path lengths up to $n-1$. We want to know whether there is a path of length 1 *or* length 2 *or* length 3 *or* ... — and each of these is represented by a matrix, while OR is $+$. So we go back to **matrix addition**:

$$A^+ = A + A^2 + A^3 + \cdots + A^{n-1}$$

For each (i, j) we are checking whether A_{ij} is 1, or A^2_{ij} is 1, or A^3_{ij} is 1, and so on up to A^{n-1}_{ij} — whether **any** of them is 1. And that is the same as doing + throughout.

The transitive closure is just the **matrix sum** of these $n - 1$ matrices — where A is really A^1 implicitly, the length-1 paths; A^2 the length-2 paths; and so on.

That is how we got from transitive closure to matrix multiplication: by using the entries of A as **Boolean values** and interpreting the two operations + and $\times$ as logical **OR** and logical **AND**.

12 Summary

Operation	Rule	Requirement
Addition $C = A + B$	defined **element-wise**: the (i, j)-th entry of the sum is the sum of the two (i, j)-th entries	A and B must have the **same** number of rows and the same number of columns
Multiplication $C = A \times B$	for each entry (i, j), go along row i of A and down column j of B, multiply pairwise, and add them all up: $C_{ij} = \sum_k A_{ik} B_{kj}$	the number of **columns** in A must equal the number of **rows** in B; the result has as many rows as A and as many columns as B

And if we interpret the adjacency matrix as a **Boolean matrix** — the values 0 and 1 representing false and true, with AND and OR represented by multiplication and addition — then the transitive closure computation becomes: first a bunch of **matrix products**, giving $A^{\ell+1} = A^\ell \times A$; and finally we **add up** all the matrices computed, from A^1 up to A^{n-1}, where n is the number of vertices in the graph.

Lecture 4 — Weighted Graphs

Edge weights; representing them in the adjacency matrix; weighted shortest paths; single source and all pairs; negative weights and negative cycles

1 Why weights?

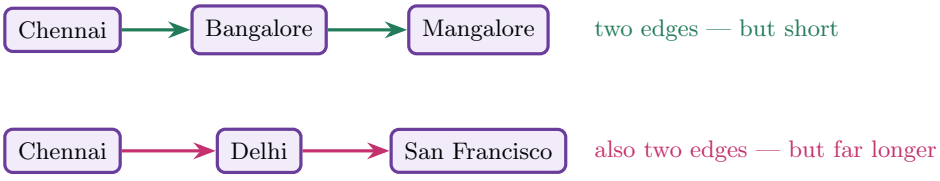
Remember that a systematic way to explore a graph is **breadth-first search**, which explores the graph **level by level**. Because it does so level by level, it discovers the vertices reachable from the starting vertex at successively longer distances — and therefore BFS computes the **shortest path in terms of the number of edges** to every reachable vertex.

In practice, though, we often **assign values to the edges**.

- On a **road map** you might see numbers against each section of road representing the **length** of the road.
- On a **railway map** or an **airline map** you might see the **time** it takes to do a segment of a journey — or the distance, or even the **cost**: how much the ticket costs.

These numbers indicate more abstract information about the "length" of an edge than just the fact that we take one edge.

It is **not enough to say that I took two flights**. If I take a hopping flight over a short distance — say Chennai to Bangalore to Mangalore — that is not the same as taking a hopping flight from Chennai to Delhi to San Francisco. Both are **two-length** travels, but one is **enormously longer** than the other in time.



2 The definition

Definition 4.1 — Weighted graph

A **weighted graph** is just a graph — a set of vertices V and a set of edges E — together with **weights**. Formally the weight is a **function**

$$w : E \longrightarrow \mathbb{R}$$

which takes every edge present in the graph and assigns it some **real number**.

On the sign of the weights

Notice that we are **not** claiming at this moment that the real number is positive or negative — we shall discuss what negative weights mean. But for most of the interesting things we can think of, the weights will be 0 or more.
We can think of a 0**-cost edge**: sometimes an edge which is not there has cost 0, and sometimes it may be there. Typically, in any reasonable scenario weights are **positive** — but we will see a situation where weights could actually be negative and still make sense.

3 Representing a weighted graph

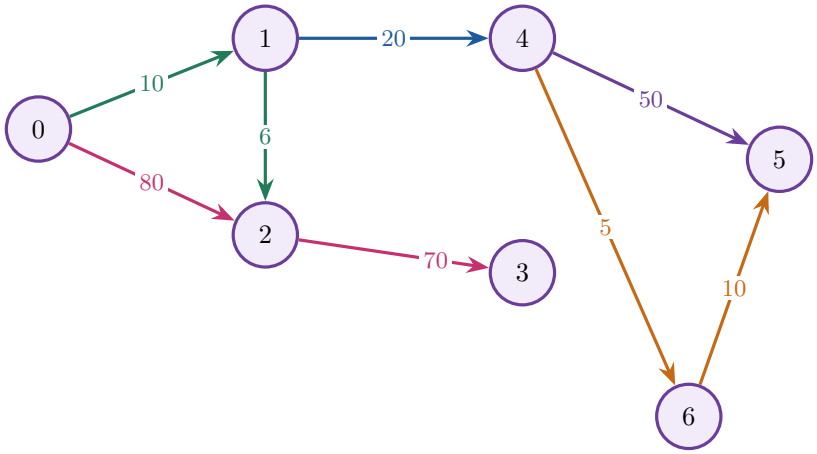
We have the adjacency matrix as a way of working with a graph in general, which records the **presence** of an edge. So how do we record a weighted graph?

Wherever we would normally put a 1 to say
there is an edge, **put the weight instead**.

Assuming there are no 0 weights: wherever there is a 0 there is **no edge**,
and wherever there is a non-zero value there is an edge **with that weight**.

So this is a very simple way to represent weighted graphs: take the adjacency matrix, and at each entry (i, j) put the **weight** of the edge (i, j); if there is no edge, or if the weight is 0, put a 0.

3.1 The example graph



Example 4.1 — reading an entry

Look at the edge from 4 to 5. In the adjacency matrix we look at the entry $(4, 5)$, and now we have a 50 there rather than just a 1.

The weighted adjacency matrix

	0	1	2	3	4	5	6
0	0	10	80	0	0	0	0
1	0	0	6	0	20	0	0
2	0	0	0	70	0	0	0
3	0	0	0	0	0	0	0
4	0	0	0	0	0	50	5
5	0	0	0	0	0	0	0
6	0	0	0	0	0	10	0

4 The weighted shortest path

Our interest is to compute **shortest paths** in such graphs. So what is the shortest path now?

Definition 4.2 — Length of a path

For us the length of a path is the **sum of the weights** of the edges traversed — not the number of edges.

Example 4.2 — three paths

$$0 \to 2 \to 3: \quad 80 + 70 = 150,$$

$$0 \to 1 \to 4: \quad 10 + 20 = 30,$$

$$4 \to 6 \to 5: \quad 5 + 10 = 15, \qquad \text{whereas the direct path } 4 \to 5 \text{ is } 50.$$

Going from 4 to 5 **via** 6 is actually **shorter** than going directly from 4 to 5.

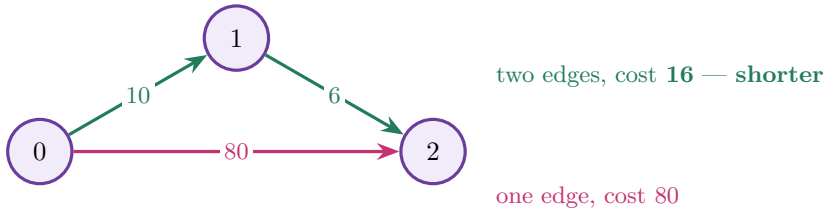
4.1 Fewest edges is not shortest

In general, in a weighted graph, the weighted shortest path **does not need to have the minimum number of edges** in the unweighted sense.

Look at the very beginning of the graph. From 0 we can go to 2 in **one step**, but then we pay a cost of 80. Whereas if we go $0 \to 1$ and then $1 \to 2$ — taking **two steps** — we get a cost of

$$10 + 6 = 16,$$

which is much less. So the shortest path from 0 to 2 is actually **via** 1: an indirect path.



5 Two shortest path problems

There is **more than one type** of shortest path problem.

5.1 Single source shortest paths

Definition 4.3 — Single source shortest path

We start at a **fixed vertex** and want to find out the distance to **every other vertex**.

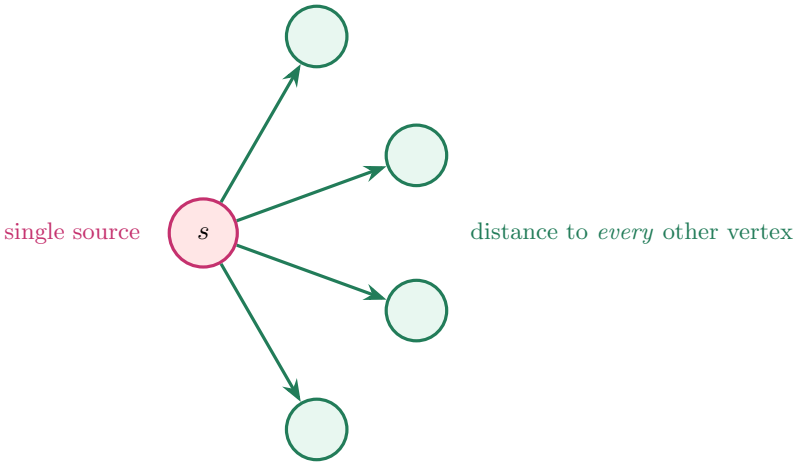
Why is this interesting? There are many applications.

Example 4.3 — a manufacturer

Suppose you are a manufacturer: you have a **factory**, you make things, and now you have to take your finished products and distribute them to the shops where they are sold — the **retail outlets**.
You have a **single source**, your factory, and you must find the most efficient way — the shortest path in terms of whatever you are measuring: the cost of travelling, the time it takes, or the distance. Whatever the cost you want to count towards the transportation cost, you would like to find the shortest transportation cost from your factory to **every one** of your retail outlets.

Example 4.4 — a courier company

A courier company has flights between cities. All the packages going to Delhi come from different destinations, land in Delhi, and go to a **centralized clearing facility**. Overnight there might be flights coming from Calcutta, Bombay, Bangalore, Chennai and so on — all these packages come to Delhi, and now they have to be delivered out.
The starting point — the distribution centre where everything is brought in from the airport or air cargo — is a **single source**, and now they must find the most efficient way to distribute to all the destinations.



5.2 All pairs shortest paths

Sometimes we want to know something about **every pair**.

Could we not just repeat the single-source version?

Of course, we could take the single-source computation and start from **every** vertex, finding every other vertex's distance — and then we would get all pairs. But generally there may be a **better way** to find the distance between all pairs.

Definition 4.4 — All pairs shortest path

For **every** i and **every** j we want to find the shortest distance from i to j.

In the single-source problem the starting point is fixed — say vertex 7 — and we ask what the shortest distance from 7 is to everything. Here it is for every i and j: not only from 7 to j, but from 9 to j, from 11 to j, from 7 to 11, from 9 to 11, and so on.

Example 4.5 — a booking site

This is the kind of thing you need if you are managing a booking site. Somebody says *I want to go from city A to city B*, and you must be able to provide — in terms of cost, or time, or some metric — the cheapest way to go from A to B.
Somebody might say they want to reach as fast as possible, so what is the shortest flight? Others might say they want the cheapest ticket. Based on whatever cost you are associating, you must find it — and you need to be able to do this for **any pair**, because you have customers who can be going from anywhere to anywhere.

Problem	What is asked	Typical application
Single source	from one fixed vertex, the distance to every other vertex	factory to retail outlets; courier distribution centre
All pairs	for every i and j, the shortest distance from i to j	a travel booking site: anywhere to anywhere

These are the two problems we will initially look at in the context of weighted graphs.

6 Negative edge weights

At the beginning we said the weight function takes every edge and gives an **arbitrary** number — and in principle there is nothing to prevent this number from being **negative**. So what would be a reasonable scenario with negative edge weights?

Example 4.6 — the taxi driver heading home

Imagine you are an Uber or Ola driver. You drive for a certain number of hours, and at a certain point you want to start **heading home** and reach home roughly when you finish work — rather than eventually travelling across the entire city empty.
(Many of us have found that towards the end of the day it is a little difficult to get long-distance rides if the driver does not live in that direction — they say *"No, sorry, I am going in the opposite direction because I am heading home."*)
So here is a taxi driver trying to head home at the end of the day. He has perhaps an **hour of service left** and wants to optimise where he lands up at the end of it. He has a minimum amount of time to travel home, and now he has to make choices.

- If he takes a road with **very few customers**, he will be **losing money** — he is paying for driving the car, paying petrol costs and other costs. That is a **positive cost**.
- If he travels on a road with **many customers**, he is likely to find somebody who will hail him for a ride, or he might get a call — so he will **earn money**. That is a **negative cost**.

He obviously wants to get more money, so he wants to **reduce the cost**: to make the negatives outweigh the positives. So he wants to find a route towards home which **minimises the cost**.

This is one example where negative edge weights make sense.

7 Negative cycles

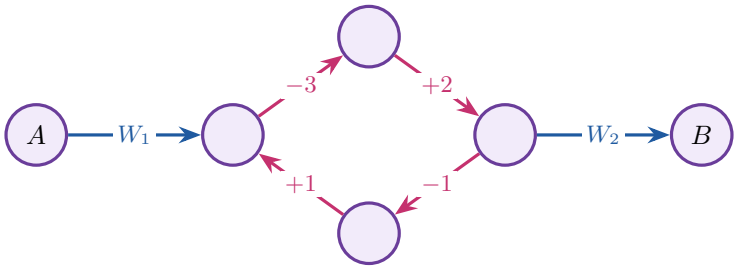
What happens to our shortest path problem in the presence of negative edge weights? The problem is not so much with negative edge weights, but with **negative cycles**.

Example 4.7 — a cycle of total weight -1

Suppose somewhere in the graph there is a cycle with weights -3, $+2$, -1 and $+1$. Going around the cycle we add up the weights:

$$-3+2=-1, \qquad -1+(-1)=-2, \qquad -2+1=-1.$$

So the **total weight of the cycle is** -1. In other words, if I go around the cycle **once**, I **reduce my cost by** 1.



total weight of the cycle $= -1$

7.1 Why this breaks the problem

Suppose I am going from A to B, with some arbitrary weights W_1 and W_2 along the way, so the cost is $W_1 + W_2$. But if I want to reduce it:

I go around the cycle **uselessly once** and I get -1; I go around it **again** and I get -2; I go around it **again** and I get -3. I can go around the cycle **as many times as I want**, and keep reducing my cost — I can make it as negative as I want.

And this does not make sense, because we are actually taking **longer and longer** paths — but because the cycle has a negative cost we are able to do this. So this clearly makes the problem **undefined**.

If we have a negative cycle we can just go round and round it, and then there are no shortest paths any more for anything which goes past that cycle — because every time we want to reduce the cost, we just go around the cycle once more.

- When a graph has **negative cycles**, shortest paths are **not defined**.
- But if we have some **negative edges** and **no negative cycles**, then it is **fine** — we can still compute shortest paths.

In that case, though, we must be careful that our **algorithm does not depend on the edges being positive**.

8 Summary

In a **weighted graph** we attach to every edge a cost — a number, which we call the **weight**. These can be described in our **adjacency matrix** by entering the cost or weight of every edge in place of a 1.

Once we have these edge weights, we can measure the **length of a path** in terms of the weight: not just how many steps we take in terms of edges, but the **total sum of the weights** across those edges. This gives a new notion of shortest path, which is probably more natural given the way we think about graphs representing spatial things — and we saw that it is **not necessarily the same** as the shortest path in terms of number of edges. We could have a shorter path which has a higher cost compared with a longer path.

There are at least two types of shortest path problem which we will find interesting:

- the **single source** problem — start at a fixed vertex and find how fast we can go to every other vertex; for example the delivery problem for a courier company;
- the **all pairs** problem — typically what you need to solve if you run a travel agency, where you must tell somebody, from anywhere to anywhere, what the best way to go is.

And finally we looked at the peculiar problem of **negative edge weights**. There are reasonable situations modelled by them; and if we still want to compute shortest paths in their presence, what we need to ensure is that there are **no negative cycles** — because with negative cycles the shortest path is not defined. If we do not have negative cycles, then even with negative edge weights we can hope to find shortest paths.

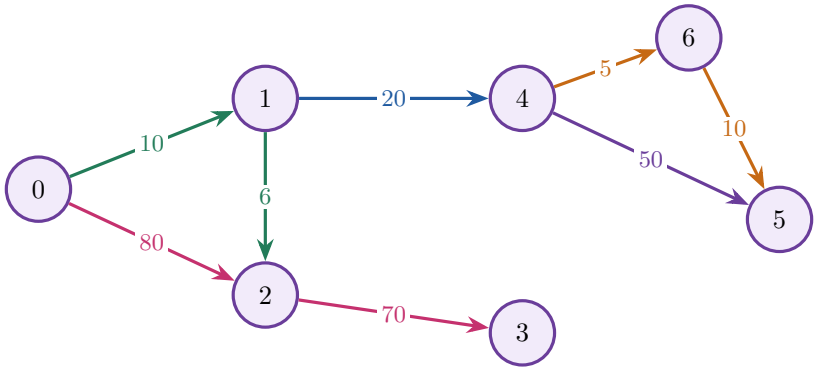
Lecture 5 — Dijkstra's Algorithm

Single source shortest paths; the burning pipeline analogy; expected burn times; why weights must be non-negative

1 The single source problem

We are looking at weighted graphs and shortest path problems on them, and we said there are two types we will focus on: the **single source** shortest path and the **all pairs** shortest path. Let us look at how we can compute single source shortest paths.

Remember that a weighted graph is just a graph with a **weight function** assigning to every edge some real number. In the graph below, for instance, the weight of the edge from 1 to 4 is 20. In the single source shortest path problem we take paths as representing the **weights** underneath — so the total length of a path is obtained by adding up the weights, not counting the edges.



We want to pick some **source** vertex — it could be any vertex — start from there, and find the shortest path to every other vertex in the graph.

> To solve this we first assume that the edge weights are all **non-negative** — we could have 0 weights, but we certainly do **not** have negative weights. Otherwise the algorithm we are going to look at will **not work**, and we will explain why.

In our example the single source is **vertex** 0.

2 The burning pipeline analogy

One way to visualise how the algorithm works is to imagine that these edges are **pipelines carrying oil**, and all the nodes are **oil depots** — storage tanks full of oil.

Now imagine what happens if we **set fire** to the source. The fire starts travelling away from 0 along the pipelines, because the pipeline also has oil: you burn the tank and the pipeline burns.

If you have ever burned firecrackers you know that it **takes time** for the fire to go along the wick. So it takes time for the fire to travel — but it travels in **all directions at the same speed**.

The fire travels towards 1 and also towards 2, but the **length of the edge** determines which one will burn next. Assume the fire moves at **one unit of length per unit of time**.

2.1 Watching it burn

- At $t = 0$ we burn vertex 0, and the fire starts moving in two directions.
- The edge to 1 is 10 units long, so in 10 units of time the fire reaches vertex 1. In those 10 units it has also travelled a short distance along the edge from 0 to 2 — since that edge is 80, the fire is only about **one eighth** of the way, nowhere near 2.
- Now that 1 is burned, fires start in two new directions. After 6 more units — at $t = 16$ — vertex 2 catches fire. The fire from 0 along the long edge is still growing but is nowhere near 2; meanwhile a fire has also started travelling towards 4.
- That fire reaches 4 at $t = 10 + 20 = 30$. By then the fire along $0 \to 2$ has covered 30 of its 80 units — about **three eighths** of the way.
- From 4 two new fires start. The edge to 6 is only 5, so in 5 more units — at $t = 35$ — vertex 6 burns.
- From 6 a **separate** fire starts towards 5, in addition to the old fire coming from 4. It arrives in 10 units, so at $t = 45$ vertex 5 burns. At that point 15 units have passed since 4 burned, so the fire along $4 \to 5$ has covered only $\frac{15}{50}$ — about one third of the way.
- Finally at $t = 86$ the fire which started along $2 \to 3$ reaches 3. At this point every vertex has burned — and here every edge has burned too, though that need not always happen.

The earliest time at which each vertex burns

vertex	0	1	2	4	6	5	3
burn time	0	10	16	30	35	45	86

These earliest times **are** the shortest paths. Notice one interesting discovery: the shortest path to 5 is **not** $0, 1, 4, 5$ but

$$0 \to 1 \to 4 \to 6 \to 5,$$

of total weight $10 + 20 + 5 + 10 = 45$, rather than $10 + 20 + 50 = 80$.

3 Turning the picture into a calculation

How do we keep track of when a pipeline is going to burn next? At every point we compute, for each vertex, the **expected time to burn**, based on the information we have so far about which pipelines are burning and which depots have burned.

The algorithm

1. Among all the vertices, the one **expected to burn next** is going to burn next — so burn it, and update its status.
2. Once a vertex burns, it starts **new fires along its edges**. So check whether any of its neighbours will now burn **faster** because of this, and update their expected burn times.

Example 5.1 — how an estimate improves

When 0 burned we started a fire along $0 \to 2$, so we expect 2 to burn at time 80. But at time 10, when 1 burns, a **new fire** starts along $1 \to 2$. So 2 is actually not going to burn at 80 — it will burn at 16.
Vertex 2 **cannot burn any later than** 16; maybe it will burn even earlier, but not later.

3.1 The trace

Initially we know nothing, so every vertex is going to burn at some indefinite time in the future — as far as we are concerned it is **never** going to burn, at time ∞. For the single source the burn time is 0, because that is when the clock starts ticking.

Expected burn times, updated step by step

After burning	0	1	2	3	4	5	6
initially	0	∞	∞	∞	∞	∞	∞
0 at $t = 0$	0	10	80	∞	∞	∞	∞
1 at $t = 10$	0	10	16	∞	30	∞	∞
2 at $t = 16$	0	10	16	86	30	∞	∞
4 at $t = 30$	0	10	16	86	30	80	35
6 at $t = 35$	0	10	16	86	30	45	35
5 at $t = 45$	0	10	16	86	30	45	35
3 at $t = 86$	0	10	16	86	30	45	35

Shaded entries are **burned** — their times are frozen.

Reading the trace:

- Having burned 0 at time 0, we look at its neighbours 1 and 2. As far as we knew, 2 was never going to burn — but now we know it will burn **at least by** time 80. Similarly 1 will burn at least by time 10. So these two entries are updated with better information.
- Now look at all the vertices and see which burns next: the **smallest** time to burn is 10, so vertex 1 burns next. Burning it starts new fires. We already believed 2 would burn at 80, but now we can say it cannot be later than 16; and 4, previously not known to burn at all, is now definitely burned by 30.
- Again look among the **unburned** vertices for the minimum. This is an **absolute** time measured from $t = 0$: not 16 more units, but $t = 16$, which is 6 units from now since we are at time 10. So 2 burns.
- When 2 burns we look at its unburned neighbours — namely 3 — and learn that 3, which we never knew would burn at all, will burn at 86. We get no new information about 5 and 6.

- The minimum among the unburned is now 4 at time 30. Burning it starts two new fires, so 6 will burn at $30 + 5 = 35$ and 5 definitely by $30 + 50 = 80$.
- The unburned times are now 35, 80 and 86, so vertex 6 burns next. It passes a new fire to 5: since 6 burned at 35, vertex 5 will burn at 45. Our previous information was 80, so we **remove the** 80 **and make it** 45 — each time we improve the estimate if we can.
- Now 5 burns; it gives no new information about 3 or anybody else. Finally 3 burns at 86.

Doing this mechanically

Although we have described it picturesquely in terms of burning vertices, all this information can be kept in a **matrix**: keep track of what is burned, and keep updating the time to burn by looking at the edges in the graph to find the neighbours, the current burning time and the expected new burning time. It is a fairly straightforward thing to do given the matrix and these burning times.

This algorithm is very well known. It is due to the computer scientist **Edsger Dijkstra**, and is called **Dijkstra's algorithm** for the single source shortest path.

4 Why does it work?

The idea is that every new shortest path we find **extends an earlier shortest path**.

The shortest path to 3 is an extension of the shortest path to 2. We get the shortest path to 2 one way; we cannot then get a shortest path to 3 which goes the other way by a longer route.

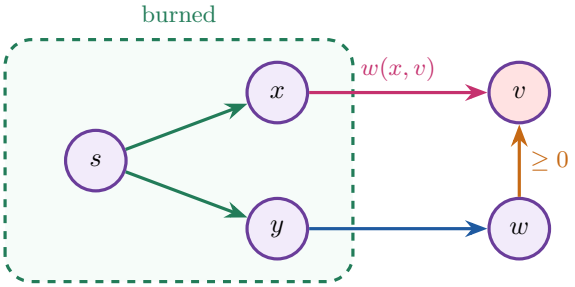
So **every prefix — every beginning segment — of a shortest path is also a shortest path**. It cannot be that we find the shortest path to a vertex, and then to go to its neighbour we use a *longer* path to that vertex.

4.1 The correctness argument

We show by **induction** that at every point, the burning times associated with burned vertices are the shortest times. The starting point is the source vertex, labelled 0 — obviously correct, since the shortest path from a vertex to itself has length 0.

At any given point some vertices are burned and some are yet to burn, each with an estimate. Say s, x and y are burned, and the next vertex to burn — the one with the shortest burning time — is v, connected to x. So we are assuming that the shortest path to x, extended by the edge xv, gives the shortest path to v:

$$\text{time}(v) \;=\; \text{time}(x) \;+\; w(x,v).$$



Is this the **final** value for v? If we choose to burn it now, are we being hasty — might we discover a better path to v later on?

Such a better path would have to come from a vertex **not yet burnt**, since among the burnt vertices we have already found the best path. So suppose that later we burn a new vertex w and through it discover a shorter path to v. Can this happen?

Why did we burn w **after** v? Because w has a burning time which is **not smaller** than v's — it could be exactly equal, but it is $\geq$. So the time at w is already equal to or greater than the time at v, and then we still have to travel the edge $w \to v$, spending some cost.

At best, by some magic, the burning time of w is exactly equal to that of v — it cannot be smaller, otherwise we would have burned w first. But even then, we could at best come from w to v at **zero cost**, because **edges are not negative**.

This is **crucial**. Without it, the decision to burn v based on its distance from x could have been **premature**.

So we cannot use Dijkstra's algorithm with negative edge weights: we might discover a long path, after burning a vertex, which becomes shorter because some negative edge weights cancel out the initial cost. It could be that w's time is bigger than v's, but because the edge is negative, coming back from w to v drops the cost of v **below** the value it had when we burnt it.

5 Summary of Lecture 5

We have an algorithm to compute **single source shortest paths**, provided the edges have **non-negative weights**.

The way to think about it is the **fire analogy**: set fire to the initial vertex, thought of as an oil tank, with the edges as pipelines. The fire spreads at a **uniform rate**, and by systematically keeping track of the time at which every vertex burns, we get the shortest path to every vertex. This is **Dijkstra's algorithm**.

Lecture 6 — The Bellman–Ford Algorithm

Dijkstra stated formally; allowing updates after burning; shortest paths by number of edges; detecting negative cycles

1 Dijkstra, stated formally

Let us recall Dijkstra's algorithm and look at it a little more formally. In the burning pipeline story, the vertices are oil depots and the edges pipelines; setting fire to a source vertex, the fire spreads at a uniform pace, and we calculate the order in which the vertices catch fire and propagate it further.

As an algorithm we keep track of which vertices have already burned — those for which we have already computed the shortest distance — and an **estimate** for the others, the **expected burn time**, which we keep updating.

Dijkstra's algorithm

Initialisation. For every vertex i, set the burn status

$$B(i) = \texttt{False}, \qquad UB = \{\, k \;:\; B(k) = \texttt{False} \,\},$$

where UB is the set of **unburned** vertices — so initially UB is the set of all vertices. Set the burning time of the source (here vertex 0) to

$$\texttt{ExpectedBurnTime}(0) = 0, \qquad \texttt{ExpectedBurnTime}(i) = \infty \text{ otherwise},$$

since we have no information about the rest.

Iteration. While there are unburned vertices:

1. pick an unburned vertex j with **minimum expected burn time**;
2. set $B(j) = \texttt{True}$ — so its expected burn time is now **frozen** — and remove j from UB;
3. for every outgoing edge (j, k) with k unburned, update

$$\texttt{EBT}(k) \;=\; \min\Big(\texttt{EBT}(k),\;\; \texttt{EBT}(j) + w(j,k)\Big).$$

That is: we compare what we **already know** about k with the **new information** obtained by processing the edge (j, k) — how much time it takes from j to k added to the time at which j burnt — and this might well turn out to be smaller.

2 Why non-negative weights were needed

We argued that we are **incrementally discovering** shortest paths: every new shortest path extends an earlier one, and inductively the burnt vertices are those whose distance has been computed and frozen, never to be updated again.

At some point we have a large set of burnt vertices — the source s, then x, y and so on. Dijkstra says: among the remaining, look for the one with the smallest expected burn time; it will be connected by an edge to some vertex in the set, so its time is the minimum of something plus the edge from x to v.

If we now burn v and **freeze** its time, we are not making a mistake — because a new path to v found later must come through another vertex w, which must also start from inside the burnt set. When we burnt v, this w had a **higher** expected burn time, otherwise we would have burnt w first. So the cost of going to w plus the cost of going from w to v cannot be less than what we already have for v — and this **crucially depends** on the edge $w \to v$ not being negative.

If it were a **large negative** edge, then going via w we could suddenly save a lot of cost. So the argument fails: we cannot freeze the cost of a vertex the first time we burn it if we are allowed to revisit it by a negative edge later on.

3 Allowing updates after burning

The difficulty is precisely that we **stop considering updates** once we burn a vertex. So what happens if we start allowing updates **even after** a vertex has burnt?

Then the notion of burning does not really make sense, so the analogy breaks down — but it is a **plausible strategy**. Why plausible? Because although we allow negative edge weights, we do **not** have any **negative cycles**.

Going from the starting vertex to some vertex x, it cannot help to go through a **loop**, because every loop is guaranteed to have non-negative weight — there are no negative cycles. So if we want the shortest path from the source to any vertex x, we may as well assume that the path is really a **path** in the strict sense: no loops, no repeated vertices.

3.1 Shortest paths by number of edges

If the path has no repeated vertices, then we can consider it in terms of the **length of the path** — the number of edges — and not only the weights. Take the shortest path from the source 0 to a vertex k:

$$0 \xrightarrow{w_1} j_1 \xrightarrow{w_2} j_2 \longrightarrow \cdots \longrightarrow j_{\ell-1} \longrightarrow k.$$

Not only is the weight minimum, but the reason we chose this path is that there are **no shorter paths with the same weight or less**.

This is not simply the fewest edges

It need not be the shortest path overall in terms of edges. We saw examples where one edge takes us with cost 80, but two edges give $10 + 6 = 16$ — so the shortest path may well be a roundabout path. What we mean is: if there is a path of length 2 which is shortest, then there is no path of length 1 achieving that cost. So ℓ is the **minimum number of edges among the minimum-weight paths**.

Now the old argument applies again. If this is the shortest path to k, could we have come to j_1 any better than by using the weight w_1? If we could come by a different route for less than w_1, then we could come to j_1 that way and continue with the same path to k — getting a shorter path to k.

So if this is the shortest path to k, it must also be the shortest path to j_1, to j_2, to j_3, and so on: **every prefix of the path is itself a shortest path**.

3.2 The idea for an algorithm

Once we have found the shortest path to $j_{\ell-1}$, we know that the update we get for k is **final** — there is not going to be a better one, because if we keep decreasing we can only decrease up to the shortest path; we cannot go below it.

And when do we hit the shortest path? When the nearest neighbour which feeds that shortest path is also frozen. So if my neighbour's shortest path is known, my shortest path will be known in the **next step**; and for that neighbour, *its* neighbour should have been known in the previous step. So we can fix these shortest paths **one at a time**: fix j_1, then j_2, then j_3, and once we fix $j_{\ell-1}$ we can fix k.

> What we want is an algorithm guaranteeing that after ℓ **updates**, all paths of length ℓ have achieved their minimum — there are no shorter-weight paths of ℓ edges or less.
>
> Then, since a path without repeated vertices has at most $n-1$ edges (once you take the n-th step you must repeat a vertex), doing this $n-1$ times finds **all** shortest paths.

A combination of two ideas

This is like a combination of **breadth-first search**, which finds shortest paths by length of path, and our **weighted** computation, which finds them by weight. We are saying that up to a given path length in terms of edges, there are no shorter paths in terms of weight.

“‘latex

4 The Bellman–Ford algorithm

The Bellman–Ford algorithm

Keep track of the distance $D(v)$ to every vertex as far as we know so far.

$$D(0) = 0 \quad \text{(the source)}, \qquad D(v) = \infty \quad \text{for every other } v.$$

Then, $n-1$ times, **blindly relax every edge**. For every directed edge (j,k), compute

$$\boxed{D_{\text{new}}(k) = \min\Big(D_{\text{old}}(k),\ D_{\text{old}}(j) + w(j,k)\Big)}$$

where all updates in one iteration are calculated using the values from the **previous iteration**. After considering every edge, we set

$$D_{\text{old}} \leftarrow D_{\text{new}}.$$

So for every directed edge (j,k), we check whether reaching k through j gives a shorter distance than the distance we currently know for k. We do this for all edges, and repeat the process $n-1$ times.

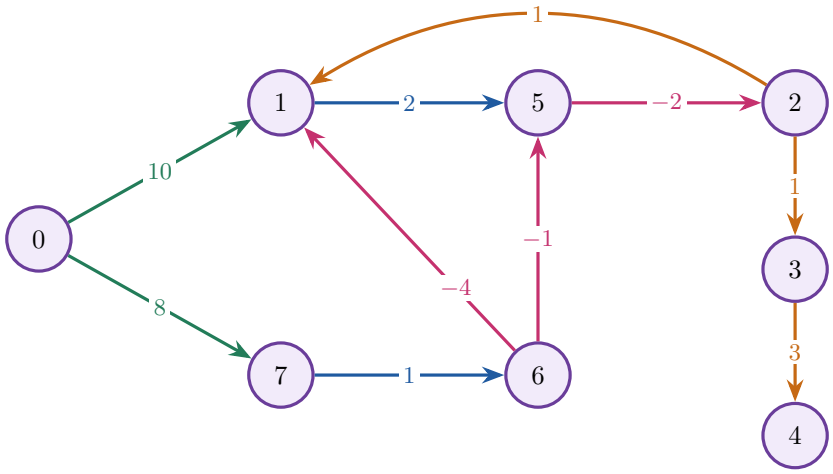
The reason $n-1$ iterations are enough is that, if there is no reachable negative-weight cycle, a shortest path can always be chosen to be a simple path. A simple path in a graph with n vertices contains at most $n-1$ edges.

It is a much simpler algorithm

In some sense this is much simpler to think about than Dijkstra — it is just a **blind iteration**. We do not have to keep track of what was burned, nor of expected burn times separately.
It works for both **directed and undirected** graphs. For an undirected graph, each undirected edge can be viewed as two directed edges, one in each direction.
Unlike Dijkstra, Bellman–Ford can handle **negative edge weights**.

5 A worked example with negative weights

This is a **directed** graph with some negative edge weights, such as -4, -1 and -2. There are arrows, so for example we can go from 0 to 1 but we cannot come back from 1 to 0.



Here $D(v)$ is the best distance we know right now for vertex v. There are 8 vertices, from 0 to 7, so $n = 8$ and we iterate $n - 1 = 7$ times after initialisation.

The Bellman–Ford table

	0	1	2	3	4	5	6	7
initial	0	∞	∞	∞	∞	∞	∞	∞
iter 1	0	10	∞	∞	∞	∞	∞	8
iter 2	0	10	∞	∞	∞	12	9	8
iter 3	0	5	10	∞	∞	8	9	8
iter 4	0	5	6	11	∞	7	9	8
iter 5	0	5	5	7	14	7	9	8
iter 6	0	5	5	6	10	7	9	8
iter 7	0	5	5	6	9	7	9	8

Shaded entries changed at that iteration.

5.1 Reading the table

- **Initialisation.** $D(0) = 0$ and everything else is ∞: we know nothing about how to reach any other vertex.
- **Iteration 1.** We look at every edge and compare the distance obtained through its starting vertex with the current distance of its ending vertex. Since only vertex 0 has a finite distance initially, the useful updates are

$$D(1) = 0 + 10 = 10, \qquad D(7) = 0 + 8 = 8.$$

- **Iteration 2.** Now we have information about vertices 0, 1 and 7. Since $D(1) = 10$, the edge $1 \to 5$ gives
$$D(5) = 10 + 2 = 12.$$
Similarly, since $D(7) = 8$, the edge $7 \to 6$ gives
$$D(6) = 8 + 1 = 9.$$

- **Iteration 3 — the strange phenomenon.** Vertex 6 was assigned 9 in the previous iteration. The negative edge $6 \to 1$ therefore gives
$$D(1) = 9 - 4 = 5.$$
So instead of going directly from 0 to 1 at cost 10, we take the **roundabout route**
$$0 \to 7 \to 6 \to 1$$
with total cost
$$8 + 1 - 4 = 5.$$

> This is exactly the kind of update that **Dijkstra's algorithm cannot handle correctly** when negative edge weights are present, because Dijkstra may permanently settle a vertex before a shorter path through a negative edge is discovered.

Also, vertex 2 can now be reached from 5:
$$12 - 2 = 10.$$
Vertex 5 itself gets a better distance because the route through 6 gives
$$9 - 1 = 8.$$

- **Iteration 4.** Using the previous iteration's value $D(2) = 10$, we can update vertex 3:
$$D(3) = 10 + 1 = 11.$$
At the same time, the improved value $D(5) = 8$ gives a better route to 2:
$$D(2) = 8 - 2 = 6.$$
Notice that this new value 6 is not used to update 3 until the **next iteration**.

- **Iteration 5.** Now the previous iteration has given us $D(2) = 6$, so
$$D(3) = 6 + 1 = 7.$$
We can also reach vertex 4 using the previous value $D(3) = 11$:
$$D(4) = 11 + 3 = 14.$$

- **Iterations 6 and 7.** We continue relaxing all edges. The distances keep improving until, after $7 = n - 1$ iterations, we obtain the stable shortest-path distances
$$\boxed{D = (0, 5, 5, 6, 9, 7, 9, 8)}.$$

> **All updates use the previous row.** Although we may calculate that the distance to 2 is 6 during one iteration, we do **not** immediately use this new value to update the distance to 3. The value $D(2) = 6$ becomes available for the next iteration.
> Thus, iteration i uses only the information produced by iteration $i - 1$. This is the **synchronous** version of Bellman–Ford used in this example.

5.2 Detecting a negative-weight cycle

After the $n-1$ iterations, we can perform one additional iteration. If any distance can still be improved, then there is a **reachable negative-weight cycle**.

Indeed, a shortest path without a negative cycle can be chosen to be simple and therefore contains at most $n-1$ edges. If a distance still decreases after that, some vertex must have been reached using a path containing a repeated vertex, which implies the presence of a negative-weight cycle. ‘‘‘

6 Detecting negative cycles

What would happen if there *were* a negative cycle? The paths would **not stabilise**: there would be a way to take a path longer than $n-1$ edges, go around the cycle, and get still shorter.

So we can run one **extra** iteration — the n-th, which we normally should not need, since we stop at $n-1$ — and see whether we get a decrease in any distance.

Check: the n-th update should **not** reduce any $D(v)$.

If it does reduce, there was a **negative cycle**. Once the $D(v)$ have stabilised there will be no further update, because the previous values simply propagate — so once the column repeats, there will be no change afterwards. If the column changes at the n-th step, you know you had a negative cycle.

7 Summary of Lectures 5 and 6

	Dijkstra	Bellman–Ford
Requires	edge weights ≥ 0	no negative *cycles* (negative edges are fine)
Strategy	burn the unburned vertex with the smallest expected burn time, then relax its outgoing edges	blindly relax **every** edge, $n-1$ times
Bookkeeping	burn status, expected burn time, set of unburned vertices	just the distance $D(v)$
Why it works	every new shortest path extends an earlier one; a later vertex w cannot improve v since w's time is $\geq v$'s and the edge is ≥ 0	every prefix of a shortest path is a shortest path, so after ℓ rounds all paths of $\leq \ell$ edges are optimal

Dijkstra's algorithm assumed non-negative weights, and the reason we needed that property was the strategy of **freezing** the distance to burnt vertices once they were burned — we never looked for updates to them, so we should not have been able to find new updates through negative edges.

Assuming we do not have negative cycles — remember that with negative cycles the notion of a shortest path is not defined, since we can go round and round and make it as short as we want — the shortest path is a genuine **path**, it cannot involve a loop, and every prefix of it is also a shortest path. Bellman–Ford uses this to iteratively find the shortest paths of length 1, of length 2, of length 3, and so on; after $n-1$ iterations we have automatically found shortest paths to everything.

The fact that shortest paths are **monotonic** — every path is an extension of a shortest path — guarantees that after $n-1$ steps all these updates converge, *unless* there is a negative cycle. So we can also use this algorithm to **find** negative cycles: go through the whole process, do it one more time, and if you find a decrease, you have a negative cycle.

Lecture 7 — All Pairs Shortest Paths

Warshall's algorithm for transitive closure; restricting the intermediate vertices; the Floyd–Warshall algorithm

1 Where we are

We are looking at shortest paths in weighted graphs, and we said there are two types of problem.

- The **single source** shortest path problem, which asks for the shortest path from a designated source vertex to every other vertex. We imagined two scenarios: a **factory** shipping finished products to retail outlets, or a **courier company** with a centralised delivery facility. Both want the shortest route from wherever they ship out to all the other locations.
- The **all pairs** problem — the shortest distance between *every* pair of vertices. This is natural for a **travel site**: somebody wants to book a ticket from somewhere to somewhere else, and you should have the shortest route regardless of which two points they are. You cannot say "I am only booking tickets from this city and not from another" — that makes no sense.

We have seen two algorithms for the single source problem: **Dijkstra's**, which works if there are no negative weights, and **Bellman–Ford**, which works even with negative weights — remembering that negative *cycles* are always forbidden, since with them the idea of a shortest path is meaningless.

> So how do we find all pairs shortest paths? We could simply **iterate** the single source computation: start from 0 and get all paths from 0, then start from 1, and so on. Doing this for every starting vertex gives the shortest path from every start to every end.
>
> But what we want to see is whether there is **another way** which does not involve going back and restarting the calculation each time.

2 Recalling the transitive closure

We begin with something we saw earlier: the **transitive closure**, discussed for unweighted graphs. The transitive closure is the **reachability** calculation — just as with all pairs shortest paths, we wanted all pairs reachability: given i and j, can I get from i to j?

Again, we could have run BFS or DFS from every starting vertex; instead we ran a **matrix** algorithm:

- the adjacency matrix represents edges, which are **paths of length** 1;
- a path of length 2 decomposes into two paths of length 1, so to find a path of length 2 from i to j we check for an intermediate k with $i \to k$ in one step and $k \to j$ in another;
- generalising, $A^\ell \times A = A^{\ell+1}$, decomposing an $\ell + 1$ step path as ℓ steps followed by one step;
- since paths are bounded in length by $n - 1$, it suffices to compute lengths 1 to $n - 1$ and take the **sum** — which was the **OR**: if A^ℓ_{ij} is 1 in *any* of these matrices, then A^+_{ij} reports 1.

3 A different induction: restricting the intermediate vertices

We now reformulate the transitive closure. Previously the induction was on the **length** of the path. Now we shall induct on **which vertices we are allowed to pass through**.

Definition 7.1 — the matrix B^k

To distinguish it from the earlier matrix, call it B. Whereas A^k_{ij} referred to a path of **length** k from i to j, here

$$B^k_{ij} = 1$$

means: **we can get from i to j visiting, in between, only vertices from the set**

$$\{0, 1, 2, \ldots, k-1\}.$$

We are *allowed*, not *obliged*, to visit them — it could be a direct edge visiting none of them, or it could use any number of them — but we **cannot** visit anything outside the set.

Two points about the definition

- Since it is a **path**, we never want to visit a vertex more than once.
- The constraint applies **only to the vertices visited in between**. So i and j need not lie between 0 and $k-1$ — i could be anything and j could be anything. We are only saying that after leaving i and before reaching j we should not see anything outside the set.

3.1 The base case

What would B^0 mean? By the definition, the intermediate vertices must lie in $\{0, \ldots, -1\}$ — and since our vertices start at 0, there is **no vertex** whose index is -1 or smaller. So no intermediate vertex can satisfy the constraint at all.

B^0_{ij} captures exactly the fact that we can go from i to j **without any intermediate vertex** — that is, there is an **edge** from i to j. So

$$\boxed{B^0 = A}$$

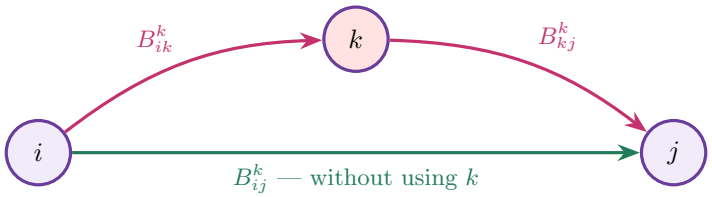
Just as before we started with paths of length 1 being precisely the adjacency matrix, here paths which pass through no intermediate vertex are precisely the adjacency matrix. The starting point of the induction is A in both cases.

3.2 The inductive step

Now we calculate B^{k+1}, which says we are allowed to use the vertices 0 to k. Going from i to j, there are **two possibilities**.

Case 1 — we do not need k at all. Then B_{ij}^k is already 1: staying within $\{0,\ldots,k-1\}$ we can already get from i to j.

Case 2 — we need to use k. There is no point visiting k more than once, so we use it exactly **once**. Then we must get from i to k, and from k to j. But if we are using up k on the way from i to k, we cannot also use k in between — so going from i to k we do not see k, and going from k to j we again do not see k. Hence both legs use only $\{0,\ldots,k-1\}$, and joining them gives a new path from i to j which visits k but does nothing else outside the set.



Warshall's algorithm

$$B_{ij}^0 = A_{ij},$$

$$\boxed{B_{ij}^{k+1} = 1 \quad \text{if} \quad B_{ij}^k = 1 \quad \textbf{or} \quad \Big(B_{ik}^k = 1 \textbf{ and } B_{kj}^k = 1\Big).}$$

This is the **more standard** algorithm found in books, and it is called **Warshall's algorithm**. We used the other formulation earlier because it is nicer for describing why it is matrix multiplication — this one requires a little more work to put in that framework, but essentially it captures the same inductive property of finding more and more complicated paths.

The **difference** between the two formulations: in the earlier one the "complication" was the **total length** of the path; here it is **which vertices** you are allowed to see in between, and that subset keeps growing until eventually you can use all of them.

3.3 When do we stop?

When the superscript reaches n, we are allowed to use any vertex from 0 to $n-1$ — that is, any vertex at all, so there is no constraint. Hence

$$B^n = A^+.$$

Just as earlier we added up from A^1 to A^{n-1}, here we go from B^0 up to B^n and we are done.

4 From reachability to shortest paths

Now we extend this to all pairs shortest paths. The correspondence is worth setting out:

	Unweighted	Weighted
Single source	BFS	Dijkstra's algorithm
All pairs	transitive closure	**all pairs shortest path**

So the transitive closure is unweighted all-pairs reachability, and we are moving to **weighted** all-pairs reachability — where, with weights, we ask for the **shortest** paths.

Definition 7.2 — the matrix SP^k

SP^k_{ij} = the **length of the shortest path** from i to j, staying within vertices $\{0, \ldots, k-1\}$ in between.

Earlier we only said *there is* a path; now, among the paths that exist, we keep track of the **minimum length**.

4.1 The base case

If there is no possibility of an intermediate vertex, we saw $B^0_{ij} = A_{ij}$. Here we do not want "edge or no edge" — we want the **weight** of the edge:

$$SP^0_{ij} = W_{ij},$$

where W is the weight matrix telling us the cost associated with each edge.

We must handle "no edge" carefully. If there is **no edge**, we assign the weight a very large number — mathematically, we treat it as ∞. So SP^0_{ij} reports ∞ if there is no edge, and the weight of the edge if there is one.

4.2 The update rule

The update rule differs from the transitive closure rule. Earlier we only needed to check *whether* one route or the other existed, so we took the logical **OR**. Here we must ask **how much** it costs to go without k, and how much we can **gain** by going with k — so we take the **minimum**.

The Floyd–Warshall update

$$\boxed{SP^{k+1}_{ij} = \min\left(SP^k_{ij},\quad SP^k_{ik} + SP^k_{kj} \right)}$$

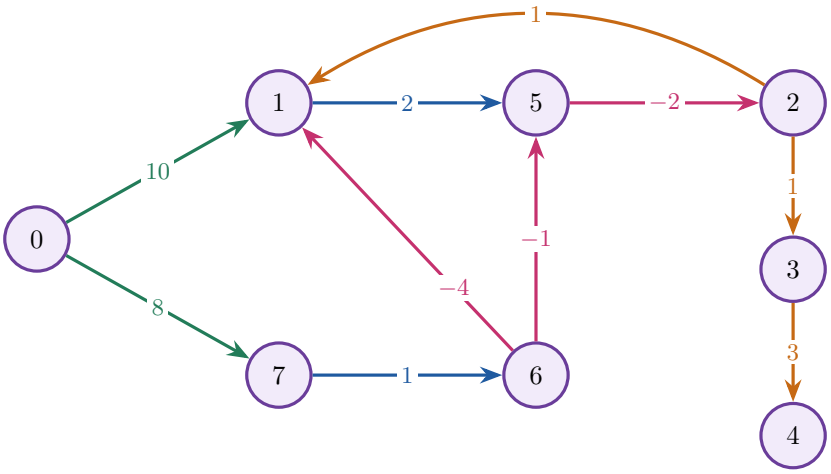
That is: either the shortest path **without** going through vertex k at all, or the shortest path from i to k followed by the shortest path from k to j — whichever is smaller.

Computing up to the n-th step gives the length of the shortest path among all paths that go through any intermediate vertex from 0 to $n-1$ — which covers **all** possible intermediate vertices. So SP^n_{ij} is the overall shortest path from i to j.

This algorithm is called the **Floyd–Warshall algorithm**. The original algorithm for transitive closure is due to **Warshall**, and **Floyd** is the person who adapted it for all pairs shortest paths — so jointly it carries both names.

5 A worked example

Let us go back to the graph we saw before — a **directed** graph with some **negative** edges.



5.1 SP^0 — the weight matrix

SP^0 is the adjacency matrix with every entry replaced by the **weight**, and ∞ where there is no edge. For instance the entry from 0 to 1 is 10. There are a great many ∞ entries and very few which are not.

$SP^0 = W$

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**
0	∞	**10**	∞	∞	∞	∞	∞	**8**
1	∞	∞	∞	∞	∞	**2**	∞	∞
2	∞	**1**	∞	**1**	∞	∞	∞	∞
3	∞	∞	∞	∞	**3**	∞	∞	∞
4	∞	∞	∞	∞	∞	∞	∞	∞
5	∞	∞	-2	∞	∞	∞	∞	∞
6	∞	-4	∞	∞	∞	-1	∞	∞
7	∞	∞	∞	∞	∞	∞	**1**	∞

5.2 SP^1 — no change at all

SP^1 represents everything reachable going **through vertex** 0. But look at 0: it has **no incoming edges**. We cannot go *to* 0 and then from 0 onwards — we can only go *out* of 0. So 0 as an intermediate vertex is useless in this graph.

SP^1 = SP^0 — the first iteration produces **no change**.

A difference from the transitive closure formulation

In the earlier formulation there were paths of length 1, then paths of length 2, and so on. Here the paths of length 2 cannot go through 0, because to do so we would have to **enter** 0 — and we cannot, since all the directions point out of 0.

5.3 SP^2 — going through vertex 1

Now we may go through $\{0, 1\}$, and in effect through 1. There *are* edges coming into 1: the three incoming edges are from **0**, **6** and **2**. These give new possibilities of going from 0, from 2 and from 6 to somewhere — but the **only** thing we can reach from 1 is **5**.

So all the updates happen in **column** 5, because that is the target.

$$0 \to 5: \quad 10+2=12, \qquad 2 \to 5: \quad 1+2=3, \qquad 6 \to 5: \quad -4+2=-2.$$

SP^2

	0	1	2	3	4	5	6	7
0	∞	10	∞	∞	∞	**12**	∞	8
1	∞	∞	∞	∞	∞	2	∞	∞
2	∞	1	∞	1	∞	**3**	∞	∞
3	∞	∞	∞	∞	3	∞	∞	∞
4	∞	∞	∞	∞	∞	∞	∞	∞
5	∞	∞	−2	∞	∞	∞	∞	∞
6	∞	−4	∞	∞	∞	**−2**	∞	∞
7	∞	∞	∞	∞	∞	∞	1	∞

5.4 SP^3 — going through vertex 2

Now that we can reach 5, we naturally want everything reachable *from* 5. From 5 we can go back to 2, and from 2 onwards to 3 and so on — so in the next round we get a lot of updates.

Example 7.1 — the entry from 5 to 1

We can now come to 5, and then go on to 1, obtaining $-\mathbf{1}$. Why? Because we can go around:

$$5 \xrightarrow{-2} 2 \xrightarrow{+1} 1, \qquad -2+1=-1.$$

SP^3

	0	1	2	3	4	5	6	7
0	∞	10	∞	∞	∞	12	∞	8
1	∞	∞	∞	∞	∞	2	∞	∞
2	∞	1	∞	1	∞	3	∞	∞
3	∞	∞	∞	∞	3	∞	∞	∞
4	∞	∞	∞	∞	∞	∞	∞	∞
5	∞	**−1**	−2	**−1**	∞	**1**	∞	∞
6	∞	−4	∞	∞	∞	−2	∞	∞
7	∞	∞	∞	∞	∞	∞	1	∞

5.5 Continuing to SP^8

In this way we keep updating. There are 8 vertices here, so if we carry this up to SP^8 we will have allowed all paths through vertices 0 to 7, and we obtain a matrix giving all the shortest paths from everywhere to everywhere.

SP^8 — all pairs shortest paths

	0	1	2	3	4	5	6	7
0	∞	5	5	6	9	7	9	8
1	∞	1	0	1	4	2	∞	∞
2	∞	1	1	1	4	3	∞	∞
3	∞	∞	∞	∞	3	∞	∞	∞
4	∞	∞	∞	∞	∞	∞	∞	∞
5	∞	-1	-2	-1	2	1	∞	∞
6	∞	-4	-4	-3	0	-2	∞	∞
7	∞	-3	-3	-2	1	-1	1	∞

A useful check

Row 0 of this matrix reads 5, 5, 6, 9, 7, 9, 8 for vertices 1 to 7 — exactly the answer Bellman–Ford produced from source 0 in the previous lecture. Column 0 is entirely ∞, since nothing can reach 0.

6 Summary

We started with **Warshall's algorithm**: an alternative, iterative way to compute the transitive closure. Earlier we did it in terms of **lengths** of paths; now we do it in terms of which **intermediate vertices** we are allowed to visit while travelling from i to j. That is the difference between Warshall's algorithm and our earlier formulation in terms of paths of length $1, 2, 3, \ldots$

B^k_{ij} is 1 if we can reach j from i using only the vertices 0 to $k-1$ in between.

When we adapt Warshall's algorithm for shortest paths, we replace the calculation of **OR** — "either I can reach it without using k, or I can reach it using k" — by **minimum**: *what is the minimum I can do without using k, and what is the minimum I can do using k?* We then use that to update the entry from i to j.

This is the **Floyd–Warshall algorithm**, and as we saw it works with **negative weights**, provided there are **no negative cycles**.

	Warshall	Floyd–Warshall
Computes	transitive closure — all pairs *reachability*	all pairs *shortest paths*
Base case	$B^0 = A$, the adjacency matrix	$SP^0 = W$, the weight matrix, with ∞ for no edge
Combining	**OR** of the two possibilities	**minimum** of the two costs
Stops at	$B^n = A^+$	SP^n, the all pairs shortest path matrix

Lecture 8 — Minimum Cost Spanning Trees

Two motivating problems; spanning trees; three facts about trees; the two strategies of Prim and Kruskal

1 A new problem on weighted graphs

We have looked at shortest paths — both the single source version and the all-pairs version, with and without negative weights. Now, still in the context of weighted graphs, we move to a different problem: computing **minimum cost spanning trees**. Let us begin with a couple of examples to motivate it.

Example 8.1 — restoring roads after a cyclone

Suppose you are in a district which has been hit by a cyclone, and many of the roads are damaged. Immediately afterwards the first priority is to restore the roads — but you also want to restore them so that **everybody can move around as quickly as possible**.
You do not want to start at one end of the district and move sequentially to the other end. Instead you want to **prioritise** the roads to be repaired, so that everybody is connected to everybody as fast as possible.

Which set of roads should you restore, so that connectivity across the district is maximally restored — rather than some parts being connected while others remain cut off?

Example 8.2 — redundancy for an internet service provider

Suppose you are an ISP providing connectivity to a large number of customers in different cities, and your customers demand **reliability**. If a cable between two cities is cut — by an accident, or by construction work — their service is cut. So you want to lay a **parallel cable**, ensuring that if one is cut the other still works.
But you do not want to spend too much: you do not want to double up *every* cable in the network, only **enough** of them so that between any two locations there is a **redundant route**.

You are not obliged to put a double cable between every pair of cities — only enough so that everyone is guaranteed to remain connected to everyone else even if one link fails.

Both of these feed into the problem of finding a **spanning tree**.

2 Spanning trees

A spanning tree essentially asks: given a graph which is **connected**, how do we retain a **minimum set of edges** so that it **remains connected**?

A minimal set of edges that keeps things connected is a **tree** — recall that a tree is a connected acyclic graph. The intuition is that if you want to connect n nodes in a minimal way, you end up connecting them so that there are **no redundant paths**, hence no cycles.

- If you **add** an edge to a tree, you add redundancy — you get a loop.
- If you **remove** an edge from a tree, it falls apart and is no longer connected.

That is exactly why it is a **minimal acyclic connected graph**.

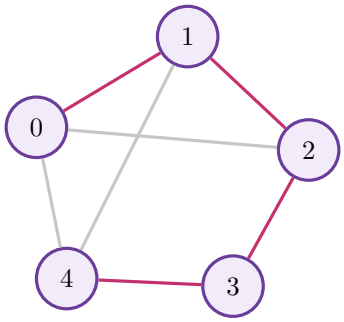
Definition 8.1 — Spanning tree

A **spanning tree** is a subset of the edges of the graph which **touches every vertex** — so it *spans* the graph — and which forms a **tree**.

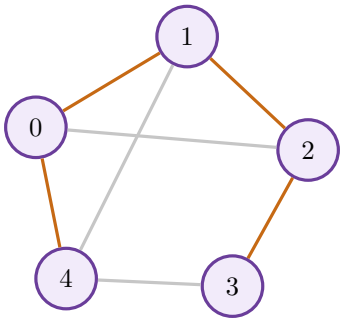
In both our situations — the roads and the ISP — we want to find a subset of the edges of the original graph such that dealing with them (repairing those roads, or upgrading those links to a double cable) connects everything to everything.

2.1 Spanning trees are not unique

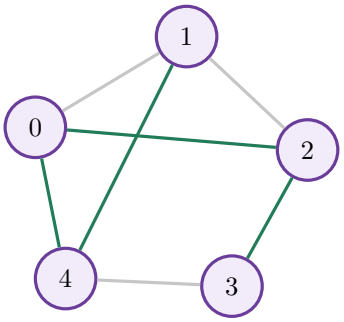
Here is a graph with three different spanning trees marked on it. Each touches all five vertices and each is a tree, so each is a valid spanning tree — there can be **many**.



one spanning tree



another one



and a third

3 Adding weights: minimum cost spanning trees

Our interest is in **weighted** graphs. Our goal is not just to find *some* subset of roads to fix or links to double up — there is a **cost** associated with each.

- Laying a road may not cost the same everywhere, depending on the location and other features.
- Similarly, there may be difficulty laying cables in some places and not in others.

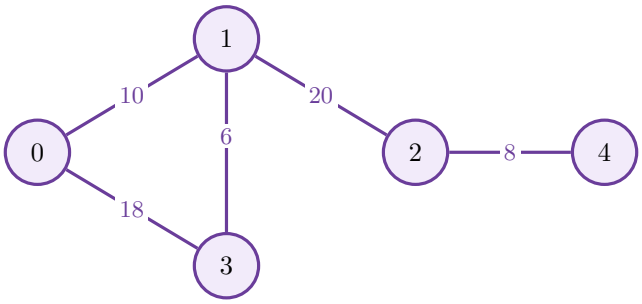
So if there is a cost or difficulty associated with every edge we want to deal with, can we find a **minimum cost** way of doing this?

Definition 8.2 — Minimum cost spanning tree

Recall that for a shortest path in a weighted graph we added the costs of all the edges on the path. Here we are constructing a **tree** in a graph, and we take **all the edges that fall into the tree**: their total is the cost we spend to build it — to repair those roads, or to lay those cables.

Among all the spanning trees, we want the one of **minimum total cost**. This is called a **minimum cost spanning tree**.

3.1 A worked example



Example 8.3 — two spanning trees of this graph

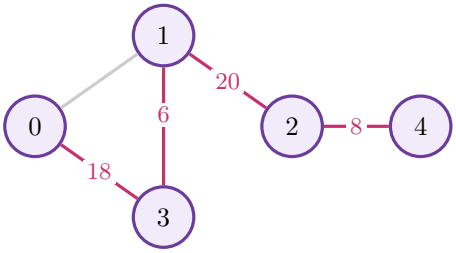
Take the spanning tree using the edges $0-3$, $1-3$, $1-2$ and $2-4$. Its cost is

$$18 + 6 + 20 + 8 \;=\; 52.$$

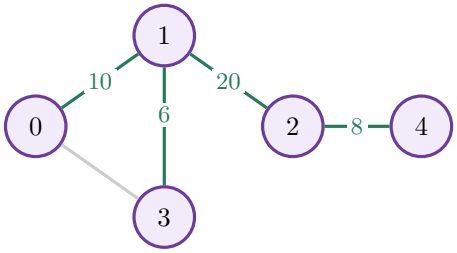
We can easily check this is **not** a minimum cost spanning tree — the graph is small enough to see it. Take out the edge $0-3$ of weight 18 and put in the edge $0-1$ of weight 10 instead. This is also a spanning tree, and its cost is

$$10 + 6 + 20 + 8 \;=\; 44.$$

Among all the trees we can draw on this particular graph, it turns out that **44** is the best.



cost $18 + 6 + 20 + 8 = \mathbf{52}$



cost $10 + 6 + 20 + 8 = \mathbf{44}$ — the minimum

4 Three basic facts about trees

To design algorithms for finding minimum cost spanning trees, we need some basic facts about trees. These are useful in general, so it is worth writing them down once and for all.

Our definition: a **tree** is a graph on n vertices which is **connected** and **acyclic**. We assume it is **undirected**, so it has no undirected cycles.

What can we conclude from this?

4.1 Fact 1: a tree on n vertices has exactly $n-1$ edges

Fact 1

If the graph has n vertices, then the tree must have **exactly** $n-1$ **edges** — not more, not less.

The argument

The graph is connected, so as an undirected graph the whole thing is a **single component**. It is also acyclic. Now:

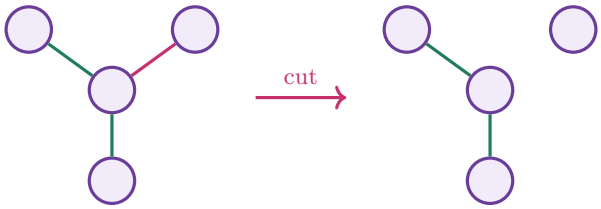
> **Claim: deleting any edge must disconnect the graph.** Suppose it did not. If I delete the edge ij and can still get from i to j by some other route, then adding the edge ij back would let me go from i to j by that other path and come back along ij — a **cycle**. But the tree is acyclic, so that is impossible.

So when we remove an edge from a tree, it falls apart into **two** components — and it cannot be more than two, because one edge only connects two parts. The whole thing was one component; it is like cutting a single thread, and the whole thing falls into two pieces.

- Now we have two connected things. Cut one more edge, and one of those two falls into two more, while the others are unchanged. So **every edge deletion creates exactly one more component**.
- How many components can we create? At most **n**, because there are only n vertices — the finest possible situation is every vertex isolated by itself with no connections.

> We start with 1 component and end with n components, adding 1 each time.
> Going from 1 to n takes $n-1$ steps — so we can delete at most $n-1$ edges.

That shows there are **no more than** $n-1$ edges. And if there were **fewer** than $n-1$, the graph would have become disconnected earlier — also a contradiction.



4.2 Fact 2: adding an edge creates a cycle

Fact 2

Whenever we **add** an edge to a tree, it **creates a cycle**.

This is symmetric to what we just said. Before, we argued that deleting an edge must split the graph into two, otherwise there would have been a cycle. Now, if we add an edge ij, we know that i and j are **already connected** in the tree — so by the same logic we have created a cycle.

> In this sense a tree is a **minimal connected graph**: adding more edges only complicates the situation in terms of connectivity.

4.3 Fact 3: there is a unique path between any two vertices

Fact 3

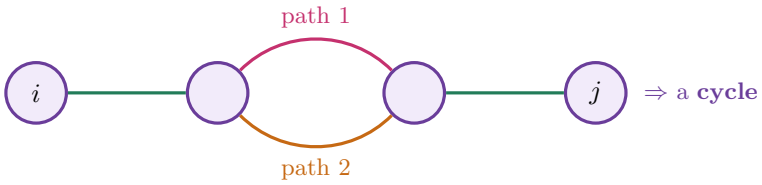
Between any two vertices in a tree there is **only one path**.

This is **not** true in general — as we have seen in many graphs you can go many ways. When calculating shortest paths we found alternative routes which gave shorter weights. But in a tree we can only go from i to j in one way: it is connected, guaranteed, but connected by **only one** way.

The argument, pictorially

Suppose there were two ways to go from i to j. Then somewhere in between there must be a structure where the two paths **diverge** and later rejoin. The two paths might diverge at i and j themselves — two completely separate paths from i to j — or somewhere in the middle.

Either way, if we can go from i one way and come back the other way, whether on the entire circuit or just somewhere in between, there must be a **cycle** we can go around. So multiple paths from i to j force a cycle to exist.



4.4 Any two of the three characterise a tree

A very useful thing to remember

Consider the three properties:

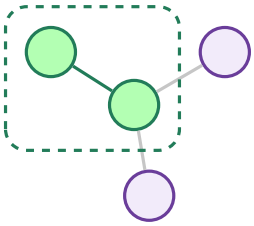
1. the graph is **connected**;
2. the graph is **acyclic**;
3. the graph has exactly $n-1$ **edges**.

If you are told **any two** of these, you may conclude that the graph is a **tree**: connected and acyclic is the definition; connected with $n-1$ edges must be acyclic; acyclic with $n-1$ edges must be connected.

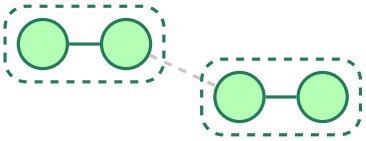
5 Two strategies

Recall the problem: a minimum cost spanning tree is a tree which touches every vertex of the given graph, taking a subset of the edges that covers all the vertices — and among those, we want the one whose sum of edge costs is minimum. There are two strategies one can think of, and we will look at an algorithm for each.

Strategy	How it proceeds
Grow one tree (Prim's algorithm)	Start from a single vertex, or the smallest single edge, and **grow a tree** incrementally. Add an edge to make a bigger tree; if adding an edge would not leave you with a tree, do not consider it.
Join components (Kruskal's algorithm)	Take a **disconnected** collection and connect it into a tree. Initially all the vertices are apart; take a small edge and connect two things; take two other vertices and connect them; then connect one group to another. You build the tree by **grouping together components** rather than growing a single tree.



Prim: one tree grows outwards



Kruskal: separate pieces get joined together

We will see both of these in detail, so that the difference between the two strategies is clear and we can see how each works.

Lecture 9 — Prim's Algorithm

Growing a tree edge by edge; the minimum separator lemma; breaking ties; why any starting vertex works

1 The setting

We are looking at minimum cost spanning trees in weighted graphs, and we said there are two natural strategies, embodied in two standard algorithms. The first is **Prim's algorithm**.

We have a weighted graph — a graph with a weight function assigning a number to every edge — and we assume the graph is **connected**. If it is not connected to begin with we cannot superimpose a tree on it: we want a spanning tree, a subset of the edges which connects the graph, and if there are not enough edges to connect it in the first place we do not even have a starting point.

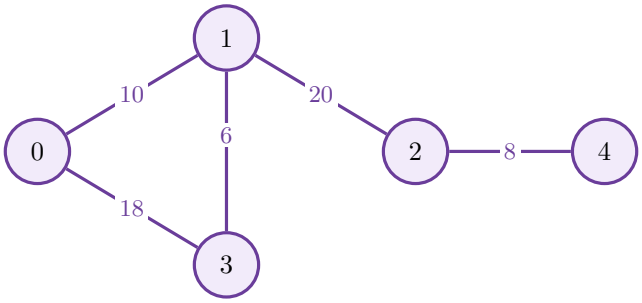
So: assuming the graph is connected and has weights, we want a **minimum cost spanning tree** connecting all the vertices in V.

The strategy

Grow the tree **incrementally**. Start with the smallest edge, and keep growing by adding the **smallest edge that can be added to the current tree while it remains a tree**.

2 A worked example

Let us take the graph we saw before.

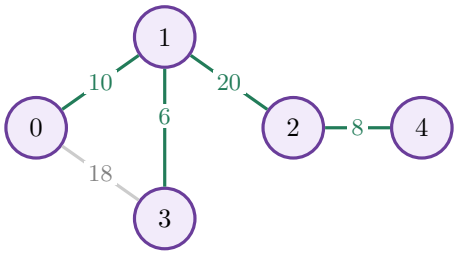


Running Prim's algorithm

Step	Edge added	Why
1	1−3 (weight 6)	the **smallest edge** in the graph — our starting tree
2	0−1 (weight 10)	the smallest edge *leaving* the tree. We may **not** take 2−4 of weight 8: it is smaller, but it is **not connected** to the tree
—	0−3 (weight 18) — **rejected**	the smallest edge next in line, but **both endpoints are already in the tree** — adding it would create a **cycle**, so we throw it away
3	1−2 (weight 20)	the next edge which leaves the tree; now four of the five vertices are connected
4	2−4 (weight 8)	now that we have reached 2, this edge *is* connected to the tree, so we may finally add it

$$\text{Total cost} \;=\; 6+10+20+8 \;=\; \boxed{44}$$

which, as we said last time, is exactly the minimum cost spanning tree for this graph.



the tree Prim builds: cost **44**

3 The algorithm formally

Prim's algorithm incrementally builds a minimum cost spanning tree. It keeps track of **two** things separately.

Why two sets? Just because a vertex has been added does not mean all the edges between the added vertices have been added. In the run above, when we had added 3 and 0, that did **not** mean the edge 0−3 was in the tree. So we must separately note which edges belong to the tree.

It is not like Dijkstra's algorithm, where we said we had *burned* these vertices and were done with them — here we need to know which **edges** were used to construct the tree.

Prim's algorithm

Keep TV, the **tree vertices**, and TE, the **tree edges**. Initially both are empty.

Start. Look at all the edges in the graph and pick the **smallest** one; call it $e = (i, j)$. Then

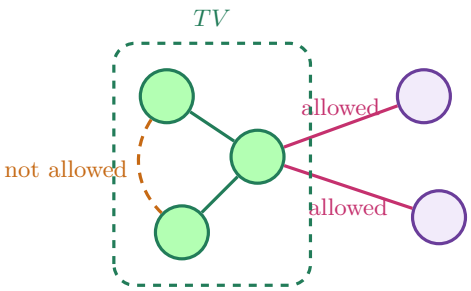
$$TV = \{i,\, j\}, \qquad TE = \{e\}.$$

Each step. Choose a **minimum weight edge** (u, v) such that

$$u \in TV \qquad \text{and} \qquad v \notin TV,$$

that is, an edge which **starts in the tree and ends outside** it. Among all such edges take the smallest and add it:

$$TV \leftarrow TV \cup \{v\}, \qquad TE \leftarrow TE \cup \{(u, v)\}.$$



The reason we could not take the edge $0-3$ in our run is exactly this: **both** its endpoints were already in the tree. We need an edge which starts inside and ends outside.

3.1 The trace in terms of TV and TE

The two sets, step by step

Step	TV	TE
initially	$\emptyset$	$\emptyset$
smallest edge overall	{1, 3}	$\{(1,3)\}$
add $(0,1)$	{0, 1, 3}	$\{(1,3), (0,1)\}$
add $(1,2)$	{0, 1, 2, 3}	$\{(1,3), (0,1), (1,2)\}$
add $(2,4)$	{0, 1, 2, 3, 4}	$\{(1,3), (0,1), (1,2), (2,4)\}$

How many steps?

We started with **two** vertices already in the tree, so there are $n-2$ vertices to go. Every time we add an edge we bring **one more vertex** into the set, so after $n-2$ steps the vertex set covers all the vertices. Here $n = 5$, and indeed we performed the step $3 = n-2$ times.

Remember that we assume the graph is **connected** — otherwise this will not work, since we would not be able to connect everything.

4 Why does this work? The minimum separator lemma

The strategy seems reasonable, but one might ask why it is *guaranteed* to give the smallest tree overall. To settle this we prove a small graph theory fact.

Partition

To **partition** the vertices into two sets U and W means: the two sets are **disjoint** and their **union** is the full set. Nothing is left out and nothing is in both.

(If I partition my toys with my sister, I take some and she takes some; each toy goes to one of us, and no toy is left out.)

Now take any partition of the vertices into U and W, and look at all the edges having endpoints on opposite sides — there may be several, say e_1, e_2, e_3, all with one endpoint in U and the other in W. Among them, suppose one is the **smallest**.

The minimum separator lemma

Let e be the **smallest edge crossing the partition** between U and W. Then **no matter how** we construct a minimum cost spanning tree on this graph, that edge e **must be in it**.

If e is the unique minimum-weight edge crossing the cut, then e belongs to every minimum spanning tree.

The intuition: somewhere in the tree we must be separating U from W — and the **best way** to connect U to W is via that smallest edge.

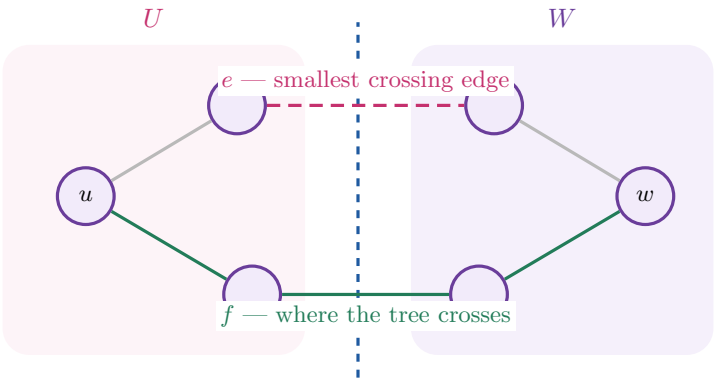
4.1 The proof

Assume for now that all the edge weights are **different** from each other; this simplifies the argument, and we deal with ties afterwards. Note that this is a **universal** property: no matter how we partition, it must hold. So take an arbitrary partition.

Suppose, for contradiction, that we have built a minimum cost spanning tree T which **excludes** the edge e that the lemma promises should be there.

T is a tree, so it is **connected** on all the vertices. Hence there is no problem going from u (in U) to w (in W) — there **must** be a path, since it is connected. That path starts in the left partition and ends in the right one.

Imagine a **river** separating two sides of a city: you cannot get from this side to that side without crossing the river. We are not using the edge e, but the path must cross *somewhere*. So there must be some **other** edge f where the path crosses.



Now see what happens. Suppose we **drop** f and instead **keep** e.

Everything that was connected before is **still connected**: anything reachable via the old crossing f can now be reached by the long path which goes across via e and comes back. So the **connectivity does not change** — but we have replaced f by a **smaller** edge.

So replacing f by e gives a **cheaper** spanning tree. This **contradicts** the assumption that we started with a minimum cost spanning tree.

So either T was not an MCST, or T was an MCST but the assumption that e did not belong to it is false. Hence:

For **any** partition of the vertex set, the **smallest edge crossing** that partition must belong to **every** MCST. If you choose that edge you are fine; if you do not, you are in trouble. ■

4.2 What if two edges have the same weight?

This is not a big problem — we just need a **strategy** to choose one or the other, arbitrarily deciding that one is smaller so that we know which will go into every tree.

Breaking ties

Number the edges in some fixed way from 0 to $m-1$. Suppose e has index i and f has index j. Then order the pairs (e,i) and (f,j) as follows:

- if $w(e) < w(f)$, declare e smaller;
- if the weights are **equal**, look at the indices: if $i < j$, then (e,i) is smaller than (f,j).

This gives a way of ordering all the equal-weight edges, and the lemma then says the tree must include the smallest *in terms of this ordering*. So it is not a big deal — **the lemma holds in general**, even if the graph has repeated weights.

5 The impact on Prim's algorithm

At any given point in Prim's algorithm we have the set of vertices in the tree, TV, and the set of vertices not in the tree,

$$W = V \setminus TV,$$

using the set minus operation — all the elements obtained by removing the elements of TV from V. Together TV and W contain all the vertices and are disjoint, so they form a **partition**.

What does Prim's algorithm do? It looks at all the edges connecting the current tree to vertices not in the tree, and **picks the smallest one**. But that is nothing but the **minimum separator** for this particular partition!

So the edge Prim picks **has to belong** to every MCST — we are never picking anything wrong.

Every edge chosen by Prim is safe: there exists an MCST containing that edge. And since it picks exactly $n-1$ edges overall, **all** of them are necessary and all are part of an MCST. That is the correctness argument.

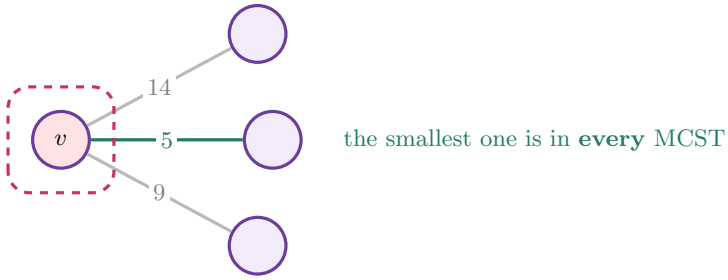
6 A stronger statement: you may start anywhere

There is a slightly stronger statement we can make. Take a single vertex v and consider the partition

$$\{v\} \qquad \text{and} \qquad V \setminus \{v\}.$$

The edges coming out of v connect it to its neighbours, and all those neighbours lie in the other part. So by the lemma, among the edges leaving v:

Among the minimum-weight edges leaving a vertex, choosing any one of them is safe: it belongs to some MCST. If the minimum edge is unique, then it belongs to every MCST



So it does not really matter that Prim's algorithm started with the minimum cost edge — that is a **bonus**. The minimum cost edge is of course the minimum separator between the partition given by its two endpoints, so that is fine; but **we can start anywhere**.

We can set TV to be a **single vertex** with no edges, and then apply Prim's algorithm. What it will first discover is the smallest edge connecting v to one of its neighbours — and by the lemma that first choice is correct.

7 Summary

Prim's algorithm is a natural way to build a minimum cost spanning tree, **starting at any vertex**. We first presented it starting from the minimum cost edge, but by the minimum separator lemma you can start at any vertex and still build an MCST.

At each step we take the tree already constructed and pick the **minimum edge connecting that tree to the rest**, extending the tree one edge at a time. At each point, because we go from **inside to outside**, the new edge is guaranteed to keep it a tree. And every edge we add is guaranteed by the minimum separator lemma to be **required** — so we have added no useless edges, and we must have obtained a minimum cost spanning tree.

Ingredient	What it says
The two sets	TV for tree vertices and TE for tree edges, kept separately, since a vertex being in the tree does not put all its edges in the tree
The step	the minimum weight edge with $u \in TV$ and $v \notin TV$ — inside to outside, so no cycle can form
Minimum separator lemma	for any partition, the smallest crossing edge is in *every* MCST; proved by swapping it for whichever edge the tree used to cross
Ties	break them by a fixed edge numbering, so the lemma still applies
Consequence	the smallest edge at any vertex is in every MCST, so Prim may start from any vertex at all

Lecture 10 — Kruskal's Algorithm

Merging components instead of growing a tree; correctness from the same lemma; repeated weights and multiple spanning trees

1 The other strategy

We are looking at minimum cost spanning trees, and we saw one strategy — **Prim's** — which incrementally **grows** a tree starting from one vertex or one edge, until we get an overall tree of minimum cost. The other strategy is **Kruskal's algorithm**.

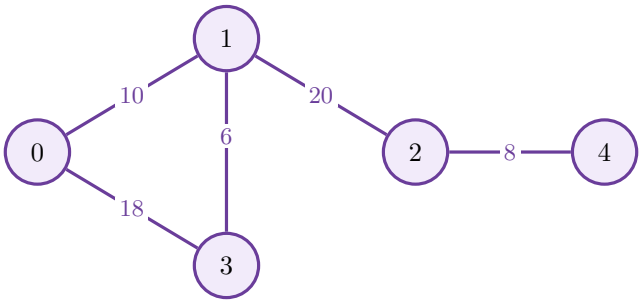
As before we work with **weighted undirected** graphs, with a weight function associating a number with every edge, and we assume the graph is **connected**. We want the minimum cost spanning tree connecting all the vertices in V.

The strategy

Start with all the vertices **disconnected**, forming n components. Then repeatedly **merge components**: connect two components by the **smallest edge** that joins them.

2 The familiar example

The first thing Kruskal's algorithm does is **sort the edges in ascending order**. Then we start with the smallest edge. Here is our familiar graph again.



Kruskal's algorithm on the familiar graph

Step	Edge	Components afterwards	What happens
—	—	$\{0\},\{1\},\{2\},\{3\},\{4\}$	five isolated vertices
1	$1-3$ (6)	$\{0\},\{1,3\},\{2\},\{4\}$	the smallest edge; now four components
2	$2-4$ (8)	$\{0\},\{1,3\},\{2,4\}$	jumps to a **different part** of the graph — allowed, since it joins two separate components
3	$0-1$ (10)	$\{0,1,3\},\{2,4\}$	still an overall disconnected graph — not a tree yet
—	$0-3$ (18) — **discarded**	unchanged	does *not* join two different components: both endpoints lie in $\{0,1,3\}$, so it forms a **cycle**
4	$1-2$ (20)	$\{0,1,2,3,4\}$	joins the last two components — done

Kruskal's rule: **include an edge if it does not create a cycle.**

2.1 The same tree, a different order

We have found **essentially the same spanning tree** as Prim's algorithm, but in a **different sequence**.

Prim	6, then 10, then 20, and finally 8 — we could only add the 8 once we had **reached one endpoint** of it, namely vertex 2
Kruskal	6, then 8 **up front**, then 10, then 20 — we create disjoint components, and as we go along these components **merge together** to form a tree

3 The algorithm formally

Kruskal's algorithm

Assume the edges are arranged in **ascending order** of weight:

$$e_0,\ e_1,\ e_2,\ \ldots,\ e_{m-1}.$$

As in Prim's algorithm we keep track of the **set of edges added**, and implicitly that set also tells us what the **components** are.

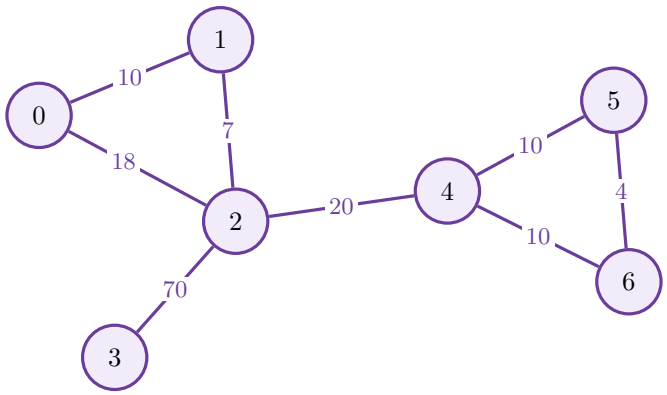
Initially the edge set is **empty**. Now scan all the edges from the smallest to the largest: if adding an edge creates a **cycle**, **skip** it; otherwise **add** it.

Components in practice

If you actually write this as code you will keep track of the components as well, which is a little tedious to program. Mathematically, though, if we know the edges we can **compute** the components just by doing reachability on each of them.

4 A larger example

Here is a slightly more complicated graph, with 7 vertices 0 to 6 and 8 edges.



4.1 Sorting, and breaking ties

We sort the edges. Here **three edges have equal weight** 10 — namely 0−1, 4−5 and 4−6 — so, as we discussed, we fix some ordering on them.

The ordering chosen here is the **lexicographic ordering of the endpoints**:

$$0{-}1 \prec 4{-}5 \prec 4{-}6,$$

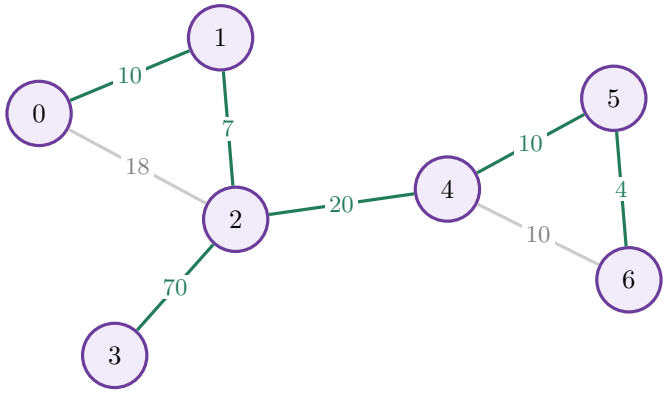
because 0 comes before 4, and 4−5 comes before 4−6 since 5 comes before 6. You may choose **any** ordering — the point is simply to fix one, so that equal-weight edges are processed in some definite order.

$$4 \prec 7 \prec \underbrace{10 \prec 10 \prec 10}_{\text{0-1, 4-5, 4-6}} \prec 18 \prec 20 \prec 70$$

4.2 The run

Processing the edges from smallest to largest

Step	Edge		Components afterwards	Decision
1	5−6	(4)	$\{5,6\}$ and five singletons	no components before this, so **add**
2	1−2	(7)	$\{1,2\},\{5,6\},\ldots$	connects two different components — **add**
3	0−1	(10)	$\{0,1,2\},\{5,6\},\ldots$	also connects two different components — **add**
4	4−5	(10)	$\{0,1,2\},\{4,5,6\},\{3\}$	$\{5,6\}$ was one component and 4 another — **add**
—	4−6	(10) — **skipped**	unchanged	now lies **within** the component $\{4,5,6\}$ — leave the edge set unchanged
—	0−2	(18) — **skipped**	unchanged	same problem: 0 and 2 are already in the same component
5	2−4	(20)	$\{0,1,2,4,5,6\},\{3\}$	takes this component and that one and connects them — **add**
6	2−3	(70)	$\{0,1,2,3,4,5,6\}$	finally brings in vertex 3 — **add**



the minimum cost spanning tree: $4+7+10+10+20+70 = 121$, using $6 = n-1$ edges

5 Why Kruskal's algorithm is correct

The reason Kruskal's algorithm is correct is **exactly the same** as the reason Prim's algorithm is correct: the **minimum separator lemma**.

Recalling the lemma

Take the graph and split the vertices into two parts U and W — **no matter how** we partition. Look at all the edges which cross between them; among these, pick the **smallest**. That smallest edge **must belong to every** minimum cost spanning tree.

5.1 Applying it to Kruskal

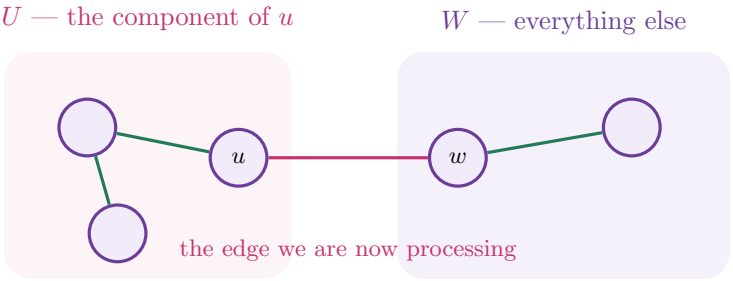
The edges found so far in Kruskal's algorithm **partition the vertices into connected components**. Initially each vertex is in a separate component; when we add an edge (u, w) it **merges** the components of u and w. We can maintain this incrementally: each time we add an edge, the component containing u now contains w as well.

- If the edge we want to add lies **within** a component — both endpoints in the same one — then it would form a **cycle**, and we discover that immediately.
- If it connects **two different** components, we apply the lemma.

Let U be the component containing u, and let W be **the rest** — so W contains w, since u and w are in different components. This is a **partition**.

Now, we are processing the edges in **ascending order**. Since u and w are still in different components, we have **not yet found any edge** between U and W — because if we had, we would already have connected those components. The reason they are not connected is precisely that we have not come to such an edge yet.

But among the edges we have *not* yet looked at, the one we are looking at **right now is the smallest**, since we are scanning in ascending order. So this edge must be the **smallest edge connecting U to W** — it satisfies exactly the property of a minimum separator.



the edge Kruskal picks is a minimum-weight edge crossing the cut,
so by the cut property it is safe: there exists an MCST containing it.

5.2 The difference between the two

Kruskal	assembles the tree **bottom up**: it takes all these disconnected things and goes around putting them together
Prim	starts somewhere and **grows** a tree gradually to cover the whole thing

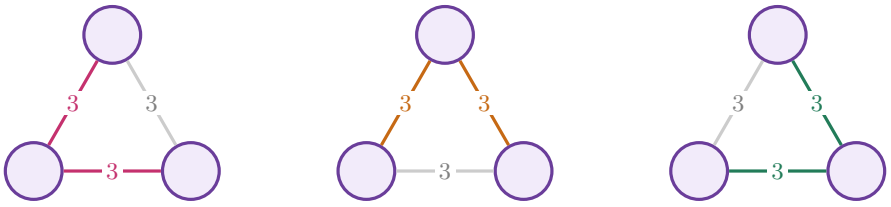
Different strategies — but both owe their correctness to **that one lemma**: whenever we partition the vertices into two disjoint sets, the smallest edge connecting the two parts must be in every MCST.

6 Repeated weights and multiple spanning trees

If there are **repeated edge weights**, we may not get a **unique** spanning tree — just as in the unweighted case there were many spanning trees.

Example 10.1 — the triangle

Take a very simple graph: three vertices, with weight 3 on each of the three edges. Then **any** pair of edges is a spanning tree — we could take this pair, or that pair, or the third pair. All of them are minimum cost spanning trees, of cost 6.



one choice another and a third

So a triangle on three vertices has **three** spanning trees.

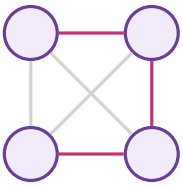
Where this shows up in the algorithm

When we say "choose the minimum cost edge", recall that with ties we specify the edge as a pair (e, i) with an index. So **the ordering we choose decides which one gets picked up** — and that is why we get different choices.

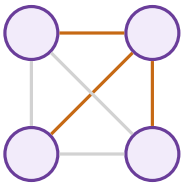
Example 10.2 — the square with both diagonals

Take a square and add the two diagonals. A spanning tree on 4 vertices has 3 edges. There are the obvious ones — three edges going around the outside — but there are also trees made of **two outside edges and a diagonal**.

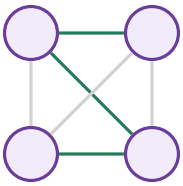
How many are there? There are several ways of going around the outside; several ways of choosing which corner to include; and the "Z" shapes can also be formed in many orientations. So with 6 edges we get a large number of spanning trees.



three outside edges



two edges and a diagonal



a "Z" shape

7 Which tree do the algorithms produce?

In general, **different choices lead to different spanning trees**. If the edge weights are not unique, there could be a very large number of them. Depending on how we have chosen to order the equal-weight edges, Prim's and Kruskal's algorithms will pick **one particular** tree out of these — they will not give you all of them, only one.

- If all the edge weights are **distinct**, then by the minimum separator lemma **every choice** in Prim's algorithm is **forced**, and every choice in Kruskal's algorithm is forced — so they give **exactly the same** spanning tree. As long as the weights are distinct, it does not matter which algorithm you use.
- If the edge weights **repeat**, then depending on how you order them, the two algorithms might produce **different** spanning trees — with the **same minimum cost** but a **different set of edges**. Keep that in mind.

8 Summary

	Prim	Kruskal
Picture	grow **one** tree outwards	merge **many** components bottom up
Start	one vertex, or the smallest edge	all n vertices isolated
Each step	smallest edge from the tree to outside it	next edge in ascending order, **skipped** if it makes a cycle
Preparation	none	**sort** all the edges first
Correctness	minimum separator lemma, with the partition TV versus the rest	minimum separator lemma, with the partition (component of u) versus the rest

References and Acknowledgement

These notes are based on the video lectures of Mathematics–1 offered by the IIT Madras BS Degree Programme.

Notes created by: Vishal (Instructor)